Содержание

Введение **4**

1 Теоретические основы и анализ архитектур детекции дисфлуенций **7**

1.1 Клинико-акустическая классификация заикания 7

1.2 Обзор существующих наборов данных и проблема доменного сдвига 8

1.3 Математический аппарат самообучающихся акустических трансформеров 9

2 Методология, инженерия данных и аппаратные оптимизации **11**

2.1 Конвейер подготовки корпуса SEP-28k и разведочный анализ 11

2.2 Математическое обоснование фокальной функции потерь (Focal Loss) 12

2.3 Офлайн-экстракция признаков и оптимизация ОЗУ методом квантования 12

2.4 Математическая модель физической кинематики лица 14

3 Архитектура нейросетевых классификаторов и мультимодальное слияние **17**

3.1 Базовый полносвязный классификатор 17

3.2 Временная сверточная сеть 17

3.3 Мультимодальная архитектура раннего слияния 18

4 Вычислительные эксперименты, метрики и анализ результатов **20**

4.1 Система метрик и проблема коллапса при оптимизации F1-меры 20

4.2 Сравнительный анализ акустических энкодеров на корпусе SEP-28k 21

4.3 Алгоритм кросс-языковой валидации и построения временных шкал 23

Заключение **24**

Список литературы **25**

Приложение А. Полные листинги программного кода **26**

Введение

В современном цифровом обществе системы автоматического распознавания речи (Automatic Speech Recognition, ASR), голосовые интерфейсы, средства телемедицины и интеллектуальные диалоговые ассистенты стали неотъемлемой частью технологического ландшафта [3, 5]. Однако для значительной доли населения, страдающей патологическими нарушениями беглости речи, эти технологии зачастую остаются малодоступными. Заикание представляет собой сложное нейромоторное нарушение речи, которое характеризуется непроизвольными спазмами артикуляционного аппарата, приводящими к внезапным остановкам (блокам), многократным повторениям отдельных звуков, слогов или целых слов, а также неестественным удлинениям (пролонгациям) фонем [2]. Стандартные акустические модели распознавания речи, обученные преимущественно на корпусах плавной дикторской речи, воспринимают подобные дисфлуенции как фоновый акустический шум, конец высказывания или неразборчивую артикуляцию, что приводит к критическим сбоям в транскрибации и семантическом анализе.

Разработка алгоритмов автоматической детекции и классификации заиканий является актуальной междисциплинарной задачей на стыке вычислительной лингвистики, цифровой обработки сигналов, компьютерного зрения и глубокого машинного обучения [1, 2]. Существующие автоматизированные детекторы, опирающиеся исключительно на акустическую модальность, сталкиваются с фундаментальным противоречием между полнотой (Recall) и специфичностью (Specificity). Модели, настроенные на высокую чувствительность, успешно находят большинство патологических эпизодов, но одновременно демонстрируют крайне низкую специфичность, ошибочно маркируя естественные интонационные паузы, вдохи, смысловые акценты и раздумья как заикание [1]. Подобная гипердиагностика делает сгенерированные временные шкалы непригодными для клинического анализа логопедами и исследователями [1].

Второй фундаментальной проблемой является языковая зависимость существующих решений. Подавляющее большинство современных открытых наборов данных (таких как SEP-28k и FluencyBank) содержат исключительно англоязычную речь [2]. Акустические трансформеры, дообучен-

ные на этих ресурсах, склонны переобучаться на специфические фонетические, интонационные и просодические паттерны английского языка [1]. При попытке прямого переноса таких моделей на русскоязычные аудио- и видеозаписи возникает выраженный эффект доменного сдвига, ведущий к резкой деградации обобщающей способности классификатора [1].

Целью данной производственной практики является комплексное исследование, математическое обоснование и программная реализация робастной мультимодальной системы автоматической детекции заиканий, объединяющей пофреймовый акустический анализ с физической кинематикой лица и способной к эффективному межъязыковому переносу (Zero-Shot Cross-Lingual Transfer) в условиях строгих аппаратных ограничений персональных вычислительных систем.

Для достижения поставленной цели в ходе работы были сформулированы и решены следующие научно-практические задачи:

1. Проведение аналитического обзора предметной области, исследование существующих корпусов дисфлуенций [2], анализ уязвимостей SOTA-архитектур (Vocametrix, StutterLens) [1] и формирование строгого математического аппарата оценки качества классификации.

2. Программная реализация автоматизированного конвейера загрузки, декодирования, ресемплинга и стандартизации крупномасштабного набора данных SEP-28k (31 908 аудиофрагментов) [2] с проведением разведочного анализа данных (Exploratory Data Analysis, EDA).

3. Разработка модулей офлайн-экстракции пофреймовых признаков с использованием трех предобученных самообучающихся трансформеров (`Wav2Vec2-Large-XLSR-53` [3], `WavLM-Base-Plus` [4], `Whisper-Small` [5]) и внедрение алгоритмов низкоуровневого квантования памяти (`float16`), позволивших сократить потребление ОЗУ с 19.6 ГБ до 9.8 ГБ.

4. Математическое обоснование и реализация архитектур временных сверточных сетей (`StutterTCN`) на базе каскада расширенных остаточных сверток (Dilated Residual Convolutions) [7] и маскированного глобального усреднения (Masked Global Average Pooling).

5. Выявление и устранение проблемы коллапса мажоритарного класса при оптимизации по F1-мере на несбалансированных выборках путем перевода критерия ранней остановки (Early Stopping) и сохранения весов на интегральную метрику ранжирования ROC-AUC в сочетании с фокальной функцией потерь (Focal Loss) [8].

6. Проектирование визуального контура извлечения 42 языконезависимых параметров физической кинематики лица на базе трехмерной сетки `MediaPipe Face Mesh` [6] с расчетом скоростей, ускорений, индекса тремора и робастной нормализацией по межквартильному размаху (IQR) [1].

7. Разработка архитектуры глубокого раннего слияния модальностей (`DeepFusionTCN`) [7] с механизмом модального дропаута (Modality Dropout) и реализация конвейера скользящего окна для кросс-языковой валидации на русскоязычных видеозаписях [1].

1. Теоретические основы и анализ архитектур детекции дисфлуенций

1.1. Клинико-акустическая классификация заикания

С точки зрения речевой патологии и цифровой обработки сигналов, проявления заикания обладают высокой гетерогенностью и затрагивают как спектрально-временные характеристики звука, так и биомеханику лицевой мускулатуры [1, 2]. В рамках компьютерного моделирования выделяются пять основных типов событий, представленных в разметке современных датасетов [2]:

- **Блоки:** Непроизвольные внезапные остановки речевого потока, вызванные тоническим спазмом мышц гортани, языка или губ. На спектрограмме блок проявляется как участок полной или почти полной тишины, которому предшествует или за которым следует резкий выброс акустической энергии. Исключительно по аудиосигналу блок крайне сложно отличить от обычной смысловой паузы, однако визуально он сопровождается выраженным напряжением губ, асимметрией челюсти или тремором [1].

- **Пролонгации:** Неестественное растягивание отдельных звуков (чаще согласных фрикативных или сонорных фонем) во времени. В спектральной области пролонгация выглядит как квазистационарный участок с неизменными формантными частотами, длительность которого существенно превышает статистическую норму для данного темпа речи [2].

- **Повторения звуков и слогов:** Клонические спазмы, приводящие к многократному циклическому воспроизведению начального звука или слога слова. Формируют характерный высокочастотный ритмический паттерн модуляций огибающей сигнала [2].

- **Повторения слов:** Неоднократное повторение целых односложных или многосложных слов, возникающее как попытка говорящего преодолеть речевой ступор [2].

- **Интеръекции:** Вводные паразитные звуки и заполненные паузы («э-э», «м-м», «ну»), используемые для маскировки спазма или отсрочки произнесения трудной фонемы [2].

Разнородность указанных классов делает неэффективным применение классических линейных моделей или простых спектральных пороговых детекторов, требуя построения иерархических моделей, способных одновременно анализировать локальную микродинамику и глобальный контекст окна [7].

1.2. Обзор существующих наборов данных и проблема доменного сдвига

Ключевым индустриальным стандартом для обучения систем детекции речевых нарушений выступает корпус **SEP-28k** (Stuttering Events in Podcasts), разработанный исследователями компании Apple и дополненный записями из корпуса **FluencyBank** [2]. Объединенный набор содержит более 31 000 трехсекундных аудиоклипов, извлеченных из англоязычных подкастов с участием людей с заиканием [2]. Среди других известных ресурсов выделяются синтетический корпус **LibriStutter** и немецкоязычный терапевтический корпус **KSOF** (Kassel State of Fluency) [1, 2].

Несмотря на большой объем SEP-28k, модели, обученные исключительно на нем, страдают от сильного доменного сдвига при переносе на другие языки [1]. В исследовании Е. Иванкина и Н. Базенкова [1] была сформирована пилотная база из 39 русскоязычных видеозаписей от 20 дикторов с заиканием, размеченная экспертами с точностью до кадра. Статистический анализ этого корпуса показал, что медианная длительность эпизодов заикания в живой русской речи составляет менее 1.0 секунды [1]. Это означает, что модели, усредняющие признаки на 3-секундном интервале, неизбежно размывают короткие патологические события на фоне окружающих их нормальных слов.

При тестировании открытых англоязычных аудиомоделей на русскоязычных записях авторами были зафиксированы следующие показатели [1]:

1. Архитектура **Vocametrix** (построенная на базе дообученного транс-

формера `wav2vec2-large-xlsr-53` [3]) продемонстрировала полноту Recall = 0.957, но критически низкую точность Precision = 0.327 и специфичность Specificity = 0.300 (F1 = 0.487, ROC-AUC = 0.701) [1]. Из-за отличий в русской интонации и фонетике модель классифицировала 70% нормальной речи как заикание.

2. Архитектура **StutterLens** показала обратный перекос: высокую специфичность Specificity = 0.886, но крайне низкую полноту Recall = 0.205 (Precision = 0.391, F1 = 0.269, ROC-AUC = 0.632), пропуская четыре из пяти реальных спазмов [1].

3. Предложенная в лабораторном исследовании схема позднего слияния (**Late Fusion**), объединяющая выходы Vocametrix с 20 лицевыми единицами действия (Facial Action Units, FAU из пакета PyFeat) через классификатор Histogram-based Gradient Boosting (HGB) с ограничением на высокую специфичность, позволила довести Specificity до 0.929 (Precision = 0.504, ROC-AUC = 0.710), но обрушила полноту до Recall = 0.195 (F1 = 0.282) [1].

Причиной падения полноты в схеме Late Fusion является то, что абстрактные эмоциональные единицы FAU, предназначенные для распознавания улыбки, гнева или удивления, слабо коррелируют с высокочастотной биомеханикой речевого спазма, а объединение на уровне готовых вероятностей не позволяет сети находить нелинейные кросс-модальные связи [1].

1.3. Математический аппарат самообучающихся акустических трансформеров

Для преодоления фонетической зависимости в настоящей работе проводится систематическое исследование трех фундаментальных акустических архитектур, различающихся парадигмой предобучения [3, 4, 5]:

1. Wav2Vec2-Large-XLSR-53 [3]. Модель принимает на вход нормированный сырой аудиосигнал $X = (x_1, \ldots, x_N)$ с частотой дискретизации 16 кГц. Многослойный сверточный энкодер $f : \mathcal{X} \mapsto \mathcal{Z}$ с шагом свертки 320 отсчетов (что соответствует временному разрешению 20 мс или частоте 50 Гц) преобразует сигнал в последовательность латентных векторов $z_1, \ldots, z_T$. Далее векторы подаются в 24-слойный блок Transformer

$g : \mathcal{Z} \mapsto \mathcal{C}$, формирующий контекстные репрезентации $c_t \in \mathbb{R}^{1024}$. В процессе самообучения на 53 языках часть векторов z_t маскируется, и модель минимизирует контрастивную функцию потерь, угадывая истинный квантованный вектор q_t среди набора негативных дистракторов $\tilde{q} \in \mathcal{Q}_t$ [3]:

$$\mathcal{L}_m = -\log \frac{\exp(\mathrm{sim}(c_t, q_t)/\kappa)}{\sum_{\tilde{q}\in\mathcal{Q}_t} \exp(\mathrm{sim}(c_t, \tilde{q})/\kappa)}, \quad \mathrm{sim}(a, b) = \frac{a^\top b}{\|a\|\|b\|} \tag{1}$$

где κ — температурный гиперпараметр. Несмотря на мультиязычность, целевая функция Wav2Vec2 ориентирована на различение фонем, что делает ее чувствительной к артикуляционным сдвигам [3].

2. WavLM-Base-Plus [4]. Архитектура от Microsoft расширяет концепцию самообучения за счет механизма Masked Speech Denoising [4]. Во время предобучения к входным высказываниям искусственно добавляются перекрывающиеся голоса других дикторов и акустические шумы, однако от трансформера требуется предсказать дискретные псевдо-метки чистого целевого сигнала на замаскированных позициях. Кроме того, в механизме самовнимания используется относительное позиционное смещение (Gated Relative Position Bias) [4]. Благодаря этому 12-слойный энкодер (`hidden_dim` = 768) сохраняет в скрытых состояниях богатую паралингвистическую информацию: тембр, микротремор голосовых связок, дыхательные паузы и акустические артефакты мышечного зажима [4].

3. Whisper-Small [5]. Модель от OpenAI обучена в режиме слабой супервизии на беспрецедентном объеме данных — 680 000 часов многоязычного и мультизадачного аудио [5]. В отличие от Wav2Vec2 и WavLM, на вход Whisper подается 80-канальная лог-мел спектрограмма, вычисленная в окнах по 25 мс с шагом 10 мс. Два входных сверточных слоя с функцией активации GELU и шагом 2 сжимают временную ось в два раза (до разрешения 20 мс, 50 фреймов в секунду), после чего тензор обрабатывается стеком блоков Transformer (`hidden_dim` = 768) [5]. Поскольку модель обучалась одновременно распознавать речь, переводить ее и детектировать активность голоса на десятках языков, ее энкодер формирует инвариантные к языку репрезентации просодии и ритмики [5].

2. Методология, инженерия данных и аппаратные оптимизации

2.1. Конвейер подготовки корпуса SEP-28k и разведочный анализ

На первом этапе работы был разработан модуль автоматизированной загрузки и подготовки данных `sep28k_data_preparation.py` [2]. Для обхода системных ошибок декодирования аудиокодеков на платформе Windows загрузка из репозитория HuggingFace (`sep28k-fluencybank-stutter-dataset` [2]) осуществлялась в режиме чтения сырых байтовых потоков (`Audio(decode=Fal` Каждый фрагмент декодировался в оперативной памяти через `io.BytesIO` и библиотеку `soundfile`. Стереофонические записи приводились к монофоническому виду путем усреднения каналов:

$$x_{\text{mono}}[n] = \frac{1}{C} \sum_{c=1}^{C} x_c[n] \tag{2}$$

При отклонении исходной частоты дискретизации f_s от целевого стандарта 16 000 Гц выполнялась полифазная передискретизация через `librosa.resample`. Бинарная целевая метка `is_stutter` присваивалась равной 1, если хотя бы один из клинических индикаторов (`Prolongation`, `Block`, `SoundRep`, `WordRep`) принимал положительное значение [2]. Всего было успешно стандартизировано и сохранено на диск 31 908 аудиофайлов в формате WAV и сформирован реестр `metadata.csv`.

С помощью разработанного скрипта `sep28k_data_analysis.py` был проведен разведочный анализ данных. Анализ выявил существенный дисбаланс классов [2]:

- Фрагменты плавной речи (класс 0): 9 579 файлов (30.02%).
- Фрагменты с наличием дисфлуенций (класс 1): 22 329 файлов (69.98%).

Среди отдельных типов нарушений зафиксировано значительное перекрытие (в одном 3-секундном окне может присутствовать как блок, так и последующее повторение звука) [2].

2.2. Математическое обоснование фокальной функции потерь (Focal Loss)

При соотношении классов 70:30 стандартная бинарная кросс-энтропия генерирует доминирующий градиент от мажоритарного позитивного класса, побуждая классификатор предсказывать наличие заикания для всех входных примеров [8]. Для решения этой проблемы во все нейросетевые модели была внедрена фокальная функция потерь [8].

Пусть $\hat{y} \in \mathbb{R}$ — выходной логит нейросети, а $p = \sigma(\hat{y}) = (1 + e^{-\hat{y}})^{-1}$ — оцененная вероятность принадлежности к классу 1. Обозначим вероятность истинного класса $y \in \{0, 1\}$ как p_t:

$$p_t = \begin{cases} p, & \text{если } y = 1 \\ 1 - p, & \text{если } y = 0 \end{cases} \tag{3}$$

Тогда классическая кросс-энтропия записывается как $\mathrm{BCE}(p_t) = -\log(p_t)$, а фокальная функция потерь вводит модулирующий множитель $(1 - p_t)^\gamma$ и балансирующий вес α [8]:

$$\mathcal{L}_{\mathrm{Focal}}(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t) = \alpha \left(1 - e^{-\mathrm{BCE}}\right)^\gamma \cdot \mathrm{BCE} \tag{4}$$

При выборе параметров $\gamma = 2.0$ и $\alpha = 0.25$ примеры, в которых сеть уже уверена ($p_t \to 1$), получают близкий к нулю вес $(1 - p_t)^2 \to 0$, тогда как сложные и ошибочно классифицируемые примеры негативного класса сохраняют высокий градиент на протяжении всего цикла обучения [8].

2.3. Офлайн-экстракция признаков и оптимизация ОЗУ методом квантования

Полноценное дообучение крупных трансформеров одновременно с классификатором требует хранения промежуточных активаций всех 12–24 слоев для вычисления обратного распространения ошибки, что потребляет свыше 16–24 ГБ видеопамяти [3, 4]. Поскольку вычислительный стенд оснащен мобильным графическим ускорителем NVIDIA GeForce RTX 4050 (6 ГБ VRAM) и 16 ГБ системной оперативной памяти, была спроектирована двухэтапная схема с офлайн-экстракцией признаков.

Веса предобученных трансформеров замораживались (`param.requires_grad = False`), модели переводились в режим инференса (`model.eval()`), и каждый аудиофайл однократно пропускался через энкодер в контексте `torch.no_grad`. В отличие от базового подхода (`extract_features.py`), где матрица скрытых состояний $H \in \mathbb{R}^{T \times D}$ сжималась вдоль оси времени в один вектор $v = \frac{1}{T}\sum_{t=1}^{T} H_t \in \mathbb{R}^D$, в секвенциальных скриптах (`extract_features_seq.py`, `extract_features_wavlm.py`, `extract_features_whisper.py`) на диск сохранялась полная двумерная матрица размерности (T, D), где $T \approx 150$ фреймов для 3-секундной записи [3, 4, 5].

Особую алгоритмическую задачу представляла экстракция признаков из модели `Whisper-Small` [5]. Архитектура Whisper жестко требует на вход аудиосигнал длительностью ровно 30 секунд (480 000 отсчетов), формируя на выходе энкодера тензор фиксированной длины $T_{\max} = 1500$ фреймов [5]. Сохранение 1500 фреймов для 3-секундного клипа привело бы к 90% избыточности (сохранению представлений нулевого паддинга). Поэтому в скрипте `extract_features_whisper.py` было реализовано динамическое вычисление реального числа информативных фреймов на основе истинной длительности сигнала L_{samples} и общего шага свертки (stride = 320):

$$T_{\text{real}} = \min\left(\left\lceil \frac{L_{\text{samples}}}{320} \right\rceil, 1500\right) \tag{5}$$

После чего на диск сохранялся усеченный срез `hidden_states[0, :real_frames, :]`.

Разрешение проблемы переполнения ОЗУ (RAM Overflow): Для обеспечения высокой скорости обучения сверточных сетей (время эпохи 23–45 секунд) все 31 908 матриц должны кэшироваться в оперативную память перед началом эпох, чтобы исключить узкое горлышко дискового ввода-вывода. Однако при стандартной одинарной точности `float32` (4 байта на элемент) суммарный объем памяти для матриц Wav2Vec2 ($T = 150, D = 1024$) составляет:

$$V_{\text{fp32}} = 31\,908 \times 150 \times 1024 \times 4 \text{ байт} = 19\,604\,275\,200 \text{ байт} \approx 19.60 \text{ ГБ} \tag{6}$$

На рабочей станции с 16 ГБ ОЗУ это неизбежно вызывало критическую ошибку нехватки памяти. Для решения проблемы в классах `SequentialDataset`

и `MeanPooledDataset` (модуль `utils.py`) было внедрено хранение массивов в формате половинной точности IEEE 754 `float16` (2 байта на элемент):

$$V_{\text{fp16}} = 31\,908 \times 150 \times 1024 \times 2 \text{ байт} \approx 9.80 \text{ ГБ} \tag{7}$$

При запросе конкретного сэмпла загрузчиком `DataLoader` в методе `__getitem__` выполняется мгновенное приведение типа `torch.tensor(self.data[idx], dtype=torch.float32)` непосредственно перед отправкой батча в видеопамять. Поскольку выходные активации нормализованных слоев трансформеров лежат в узком динамическом диапазоне $[-10, 10]$, переход к `float16` при хранении не вносит численной погрешности в обучение, но сокращает потребление ОЗУ ровно в два раза.

2.4. Математическая модель физической кинематики лица

Для замены нерелевантных эмоциональных признаков FAU [1] был разработан модуль `extract_face_kinematics.py`, использующий нейросетевой детектор `MediaPipe Face Mesh` [6] для извлечения трехмерных координат $P_i = (x_i, y_i, z_i)$ 478 анатомических точек лица с целевой частотой 15 кадров в секунду (`target_fps = 15`).

Чтобы обеспечить инвариантность признаков к расстоянию от человека до камеры и разрешению видео, в каждом кадре вычисляется базовый масштаб лица S_{face} как евклидово расстояние в $\mathbb{R}^3$ между переносицей ($idx = 6$) и кончиком носа ($idx = 1$) [6]:

$$S_{\text{face}} = \|P_6 - P_1\|_2 = \sqrt{(x_6 - x_1)^2 + (y_6 - y_1)^2 + (z_6 - z_1)^2} \tag{8}$$

На основе координат ключевых точек рассчитываются 7 базовых биомеханических параметров кадра:

1. **Коэффициент раскрытия рта:** отношение среднего вертикального расстояния между тремя парами внутренних точек верхней ($U = \{82, 13, 312\}$) и нижней ($L = \{87, 14, 317\}$) губы к горизонтальному расстоянию между уголками рта (61 и 291):

$$\text{MAR} = \frac{\frac{1}{3}\sum_{k=1}^{3} \|P_{U_k} - P_{L_k}\|_2}{\max(\|P_{61} - P_{291}\|_2, 10^{-6})} \tag{9}$$

2. **Растяжение уголков губ:** $\mathrm{LCS} = \|P_{61} - P_{291}\|_2 / S_{\text{face}}$.

3. **Вертикальное смещение челюсти:** нормированная координата подбородка ($idx = 152$): $\mathrm{JD} = y_{152} / S_{\text{face}}$.

4. **Напряжение бровей:** нормированное сближение внутренних краев бровей (70 и 300): $\mathrm{BT} = \|P_{70} - P_{300}\|_2 / S_{\text{face}}$.

5. **Высота бровей:** среднее вертикальное расстояние от бровей до верхних век глаз (159 и 386):

$$\mathrm{BH} = \frac{(y_{70} - y_{159}) + (y_{300} - y_{386})}{2S_{\text{face}}} \tag{10}$$

6. **Апертура глаз:** среднее раскрытие левого $(159, 145)$ и правого $(386, 374)$ глаза, фиксирующее сопутствующее заиканию зажмуривание и тики:

$$\mathrm{EA} = \frac{\|P_{159} - P_{145}\|_2 + \|P_{386} - P_{374}\|_2}{2S_{\text{face}}} \tag{11}$$

7. **Латеральная асимметрия челюсти:** разница расстояний от левого (234) и правого (454) краев нижней челюсти до подбородка (152), выявляющая боковые спазмы:

$$\mathrm{JA} = \frac{\big|\|P_{234} - P_{152}\|_2 - \|P_{454} - P_{152}\|_2\big|}{S_{\text{face}}} \tag{12}$$

Кадры, на которых лицо кратковременно не было обнаружено детектором, восстанавливаются методом покоординатной линейной интерполяции [1]. Далее для каждого видеоролика индивидуально выполняется робастная нормализация каждого признака f_j по медиане и межквартильному размаху ($\mathrm{IQR} = Q_{75} - Q_{25}$) с усечением выбросов на уровне $[-6, 6]$ [1]:

$$\tilde{f}_j(t) = \mathrm{clip}\left(\frac{f_j(t) - \mathrm{median}(f_j)}{Q_{75}(f_j) - Q_{25}(f_j)}, -6, 6\right) \tag{13}$$

Это полностью устраняет влияние индивидуальных анатомических пропорций лица конкретного пациента [1].

На заключительном этапе для каждого из 7 нормированных признаков при шаге дискретизации $\Delta t = 1/\mathrm{FPS}$ вычисляются 5 динамических характеристик:

- Первая производная (скорость): $v_j(t) = (\tilde{f}_j(t) - \tilde{f}_j(t-1))/\Delta t$.
- Вторая производная (ускорение): $a_j(t) = (v_j(t) - v_j(t-1))/\Delta t$.
- Абсолютная скорость: $|v_j(t)|$.
- Скользящая кинетическая энергия в окне W длительностью 0.3 с: $E_j(t) = \frac{1}{|W|} \sum_{\tau \in W(t)} |v_j(\tau)|$.
- Индекс мышечного тремора (стандартное отклонение скорости в окне 0.3 с):

$$\mathrm{Tremor}_j(t) = \sqrt{\frac{1}{|W|} \sum_{\tau \in W(t)} \big(v_j(\tau) - \bar{v}_j(t)\big)^2} \tag{14}$$

Конкатенация базовых значений и пяти производных формирует итоговый пофреймовый вектор размерности $7 \times 6 = 42$ признака.

3. Архитектура нейросетевых классификаторов и мультимодальное слияние

3.1. Базовый полносвязный классификатор

В качестве отправной точки исследования (Baseline) в скрипте `train_mlp.py` была реализована трехслойная полносвязная сеть `AudioClassifier`, принимающая на вход усредненный по времени вектор размерности $D = 1024$ [3]. Архитектура описывается последовательностью преобразований:

$$x \in \mathbb{R}^{1024} \xrightarrow{\text{Linear+BN+ReLU+Drop(0.4)}} \mathbb{R}^{512} \xrightarrow{\text{Linear+BN+ReLU+Drop(0.3)}} \mathbb{R}^{128} \xrightarrow{\text{Linear}} \mathbb{R}^{1} \tag{15}$$

Обучение проводилось оптимизатором AdamW (lr $= 10^{-4}$, weight_decay $= 10^{-3}$) с функцией потерь Focal Loss [8] и адаптивным планировщиком `ReduceLROnP` уменьшающим шаг обучения в 2 раза при отсутствии роста валидационного ROC-AUC в течение 2 эпох.

3.2. Временная сверточная сеть

Для сохранения временной структуры сигнала в модуле `train_tcn.py` была разработана архитектура `StutterTCN`, основанная на принципах построения временных сверточных сетей [7]. Входная матрица $X \in \mathbb{R}^{B \times T \times D}$ транспонируется к виду (B, D, T) и проходит через точечную свертку с батч-нормализацией и активацией ReLU, проецируясь в пространство размерности $H = 256$.

Ядром сети выступает стек из $K = 4$ остаточных блоков `ResidualBlock` с экспоненциально растущим коэффициентом дилатации $d_i = 2^i \in \{1, 2, 4, 8\}$ и размером ядра свертки $k = 5$ [7]. Внутри каждого блока одномерная расширенная свертка определяется как [7]:

$$y[t] = \sum_{m=0}^{k-1} w[m] \cdot x\left[t + d \cdot (m - \lfloor k/2 \rfloor)\right] \tag{16}$$

За счет симметричного паддинга $p = (k-1) \cdot d/2$ длина последовательности T сохраняется неизменной, а выход блока вычисляется с остаточной связью

[7]:

$$Z_{\text{out}} = \text{ReLU}\Big(Z_{\text{in}} + \text{BN}_2\big(\text{Conv}_{1\times1}(\text{Dropout}(\text{ReLU}(\text{BN}_1(\text{Conv}_{k,d}(Z_{\text{in}})))))\big)\Big) \tag{17}$$

Эффективное рецептивное поле стека из 4 блоков при $k = 5$ составляет [7]:

$$\text{RF} = 1 + (k-1)\sum_{i=0}^{3} 2^i = 1 + 4 \times (1+2+4+8) = 61 \text{ фрейм} \tag{18}$$

При разрешении 20 мс на фрейм один нейрон верхнего слоя охватывает контекст шириной 1.22 секунды, что идеально соответствует характерной длительности речевых блоков и пролонгаций [1, 2].

Для агрегации признаков переменной длины применяется маскированное глобальное усреднение. Если клип короче максимальной длины $T_{\max} = 150$, бинарная маска $M \in \{0,1\}^{B\times T}$ обнуляет вклад паддинга:

$$c = \frac{\sum_{t=1}^{T_{\max}} H_t \odot M_t}{\max\left(\sum_{t=1}^{T_{\max}} M_t, 1\right)} \in \mathbb{R}^{256} \tag{19}$$

Финальный вектор c классифицируется двухслойной головой $256 \to 128 \to 1$.

3.3. Мультимодальная архитектура раннего слияния

В модуле `train_deep_fusion.py` реализована архитектура `DeepFusionTCN`, объединяющая аудиопоток $X_a \in \mathbb{R}^{B\times T\times D_a}$ и синхронизированный поток лицевой кинематики $X_v \in \mathbb{R}^{B\times T\times 42}$ до этапа классификации [1, 7]. Поскольку частота кадров видео (15 Гц) и фреймов аудио (50 Гц) различаются, в классе `MultimodalDataset` выполняется линейная интерполяция более короткого ряда к общей временной сетке $T_{\text{target}} = \max(T_a, T_v)$, после чего последовательности нарезаются скользящим окном длиной 450 отсчетов с шагом 150. Окно получает позитивную метку, если доля размеченных заиканием фреймов внутри него составляет не менее 30% (`stutter_ratio >= 0.3`).

Аудиопоток кодируется 4-блочным энкодером `StreamEncoder` ($d \in \{1,2,4,8\}$) в тензор $F_a \in \mathbb{R}^{B\times 256\times T}$, а видеопоток — 3-блочным энкодером ($d \in \{1,2,4\}$)

в тензор $F_v \in \mathbb{R}^{B\times 256\times T}$ [7].

Механизм модального дропаута: Чтобы предотвратить переобучение сети исключительно на более размерный акустический канал, во время тренировки с вероятностью $p_{\text{mod}} = 0.2$ генерируется случайная величина $r \sim \mathcal{U}(0,1)$, и один из потоков принудительно зануляется:

$$(\tilde{F}_a, \tilde{F}_v) = \begin{cases} (\mathbf{0}, F_v), & \text{если } r < 0.1 \\ (F_a, \mathbf{0}), & \text{если } 0.1 \le r < 0.2 \\ (F_a, F_v), & \text{если } r \ge 0.2 \end{cases} \tag{20}$$

После этого представления конкатенируются вдоль канальной оси в тензор $F_{\text{cat}} = [\tilde{F}_a; \tilde{F}_v] \in \mathbb{R}^{B\times 512\times T}$, сжимаются проекцией $512 \to 256$ и обрабатываются двумя совместными блоками `fusion_blocks`, которые выявляют кросс-модальные корреляции (например, одновременное наступление акустической тишины и всплеск тремора нижней челюсти). Для честной оценки на видеоданных как в `train_face_tcn.py`, так и в `train_deep_fusion.py` применяется групповая кросс-валидация `GroupKFold` по идентификаторам пациентов, исключающая попадание одного и того же диктора в обучающую и валидационную выборки [1].

4. Вычислительные эксперименты, метрики и анализ результатов

4.1. Система метрик и проблема коллапса при оптимизации F1-меры

Для всесторонней оценки бинарных классификаторов на каждом шаге валидации вычисляется матрица ошибок: число истинно-положительных (TP), ложно-положительных (FP), истинно-отрицательных (TN) и ложно-отрицательных (FN) предсказаний, на основании которых рассчитываются [1]:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{Specificity} = \frac{TN}{TN + FP} \tag{21}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \tag{22}$$

а также порогонезависимые метрики качества ранжирования вероятностей: площадь под рабочей характеристикой приемника (**ROC-AUC**) и средняя точность (**Average Precision, AP**) [1]:

$$\text{AP} = \sum_n (\text{Recall}_n - \text{Recall}_{n-1}) \cdot \text{Precision}_n \tag{23}$$

Экспериментальное обнаружение коллапса модели по F1-мере: В первой серии запусков обучения `StutterTCN` на эмбеддингах Wav2Vec2 сохранение лучшего чекпоинта и условие ранней остановки опирались на максимизацию метрики F1. Поскольку доля позитивного класса в SEP-28k равна $p_{\text{pos}} \approx 0.70$ [2], тривиальный классификатор, константно выдающий ответ «1» для любого входа ($\text{Recall} = 1.0, \text{Precision} = 0.70, \text{Specificity} = 0.0$), автоматически получает:

$$\text{F1}_{\text{trivial}} = \frac{2 \times 0.70 \times 1.0}{0.70 + 1.0} = 0.8235 \tag{24}$$

В результате на 9-й эпохе первого запуска скрипт зафиксировал как «лучшую» вырожденную модель с F1 = 0.8251, у которой полнота составля-

ла 0.9906, но специфичность упала до катастрофических 0.0428 (при низком ROC-AUC = 0.6051), проигнорировав сбалансированную 16-ю эпоху (Specificity = 0.6279, ROC-AUC = 0.7845, но F1 = 0.8010).

Учитывая выводы лабораторной статьи о приоритете специфичности и недопустимости гипердиагностики [1], во всех тренировочных модулях (`train_mlp.py`, `train_tcn.py`, `train_face_tcn.py`, `train_deep_fusion.py`) критерий сохранения чекпоинта был заменен на максимизацию **ROC-AUC**. Так как константное предсказание дает ROC-AUC = 0.50, этот критерий математически гарантирует выбор модели с наилучшей разделяющей способностью между обоими классами.

4.2. Сравнительный анализ акустических энкодеров на корпусе SEP-28k

После модификации критерия чекпоинтинга было проведено полное обучение четырех конфигураций на едином стратифицированном разбиении корпуса SEP-28k (25 526 обучающих и 6 382 валидационных клипа, `random_state = 42`) [2]:

1. **MLP (Wav2Vec2 mean-pooled) [3]:** обучение в течение 40 эпох (по 2 с на эпоху). Оптимум достигнут на 33-й эпохе с ROC-AUC = 0.7673.

2. **TCN (Wav2Vec2) [3, 7]:** число обучаемых параметров — 1 874 945. Обучение продолжалось 49 эпох (сработал Early Stopping после пика на 39-й эпохе), достигнув ROC-AUC = 0.7935.

3. **TCN (WavLM) [4, 7]:** экстракция признаков заняла 15 мин 14 с (скорость 34.88 ф/с). Число параметров TCN — 1 809 409 (при `input_dim` = 768). Сеть сошлась значительно быстрее: пик достигнут на 7-й эпохе (ROC-AUC = 0.8143, время эпохи 23 с).

4. **TCN (Whisper) [5, 7]:** экстракция признаков из-за 30-секундного входного окна заняла 1 час 3 мин 31 с (8.37 ф/с). Однако при обучении TCN уже на 1-й эпохе модель показала качественный скачок до ROC-AUC = 0.8488 и AP = 0.9308.

Результаты независимой единой валидации всех сохраненных чекпоинтов с помощью скрипта `compare_encoders.py` сведены в Таблицу 1.

Таблица 1: Итоговое сравнение акустических архитектур на валидационной выборке SEP-28k ($N = 6382$)

Архитектура и энкодер	ROC-AUC	AP	F1	Precision	Recall	Specificity
MLP (Wav2Vec2 mean-pooled) [3]	0.7673	0.8779	0.8161	0.8007	0.8321	0.5172
TCN (Wav2Vec2-XLSR-53) [3, 7]	0.7935	0.8976	0.7932	**0.8447**	0.7476	**0.6795**
TCN (WavLM-Base-Plus) [4, 7]	0.8143	0.9105	0.8439	0.8060	0.8856	0.5031
TCN (Whisper-Small) [5, 7]	**0.8488**	**0.9308**	**0.8552**	0.8218	**0.8914**	0.5496

Полученные экспериментальные данные позволяют сформулировать три ключевых вывода:

- **Превосходство секвенциального анализа над усреднением:** на идентичных признаках Wav2Vec2 [3] замена полносвязного перцептрона (MLP) на временную сверточную сеть (TCN) [7] повысила специфичность с 0.5172 до 0.6795 (+16.23 п.п.), а точность — с 0.8007 до 0.8447. Расширенные свертки успешно улавливают локальные границы речевых блоков, не позволяя нормальным участкам клипа маскировать патологию.

- **Роль паралингвистического предобучения:** переход от Wav2Vec2 [3] к WavLM [4] дал прирост ROC-AUC на +2.08 п.п. (до 0.8143) и F1-меры на +5.07 п.п. (до 0.8439), подтвердив важность деноизинга при предобучении для сохранения признаков голосового напряжения [4].

- **Доминирование мультиязычного энкодера Whisper:** комбинация `TCN + Whisper` [5, 7] заняла первое место по четырем главным метрикам одновременно (ROC-AUC = 0.8488, AP = 0.9308, F1 = 0.8552, Recall = 0.8914), опередив базовый MLP на +8.15 п.п. по ROC-AUC. Это делает Whisper оптимальным акустическим ядром для кросс-языкового переноса [1, 5].

4.3. Алгоритм кросс-языковой валидации и построения временных шкал

Для оценки обученных моделей на длинных неразрезанных русскоязычных видеозаписях разработан модуль `evaluate_crosslingual.py` [1]. В от-

личие от классификации отдельных 3-секундных клипов [2], здесь решается задача покадровой сегментации [1].

Извлеченная последовательность признаков длины T сканируется скользящим окном длины W_{len} с перекрытием (шаг stride $= W_{\text{len}}/2$ для аудио-TCN и 150 кадров при окне 450 для Deep Fusion). Для каждого положения окна $[w_s, w_e)$ модель вычисляет вероятность заикания $p_w = \sigma(\text{logit}_w)$, которая прибавляется ко всем кадрам внутри данного интервала. Итоговая оценка вероятности для каждого кадра $t \in [0, T-1]$ вычисляется путем усреднения по всем перекрывающимся окнам, накрывшим этот кадр:

$$P_{\text{frame}}[t] = \frac{\sum_{w:t\in[w_s,w_e)} p_w}{\max\big(\text{count}[t], 1\big)} \tag{25}$$

Данный механизм сглаживает краевые эффекты сверточных окон и позволяет строить наглядные диагностические графики [1], на которых поверх кривой вероятности $P_{\text{frame}}[t]$ отображаются зеленые зоны истинной экспертной разметки и красные заштрихованные интервалы автоматической детекции.

Заключение

В результате выполнения производственной практики была спроектирована, математически обоснована и программно реализована полнофункциональная модульная система для автоматической мультимодальной детекции заиканий.

Основные итоги и достижения проделанной работы заключаются в следующем:

1. Сформирован и стандартизирован локальный корпус из 31 908 аудиозаписей SEP-28k [2]; проведен разведочный анализ, обосновавший применение фокальной функции потерь [8].

2. Разработана архитектура офлайн-экстракции пофреймовых признаков для трансформеров Wav2Vec2 [3], WavLM [4] и Whisper [5] с внедрением механизма квантования кэша ОЗУ в формат `float16`, что снизило потребление памяти с 19.6 ГБ до 9.8 ГБ и обеспечило стабильное обучение на системе с 16 ГБ RAM и 6 ГБ VRAM.

3. Выявлен и аналитически доказан эффект коллапса мажоритарного класса при сохранении весов по F1-мере на несбалансированных данных; осуществлен переход на критерий максимизации ROC-AUC [1].

4. Доказано преимущество временных сверточных сетей [7] над усреднением признаков: специфичность классификации на базе Wav2Vec2 выросла с 0.5172 до 0.6795.

5. В ходе сравнительного тестирования трех трансформеров установлено, что наилучшим качеством разделения обладает энкодер `Whisper-Small` [5] в связке с TCN [7], достигший рекордных показателей ROC-AUC = 0.8488, AP = 0.9308 и F1 = 0.8552.

6. Реализованы модули извлечения 42 признаков физической кинематики лица (`MediaPipe Face Mesh` [6]), раннего глубокого слияния с модальным дропаутом [7] и покадровой кросс-языковой валидации со скользящим окном [1].

Созданная кодовая база полностью готова к финальному развертыванию и тестированию на русскоязычных видеозаписях для клинической поддержки принятия решений [1].

Список литературы

[1] Ivankin E., Bazenkov N. *Audio-Visual Stuttering Detection in Russian Video Recordings.* Moscow Institute of Physics and Technology, 2026.

[2] Lea C., Mitra V., Joshi A., Kajarekar S., Bigham J. P. *SEP-28k: A Dataset for Stuttering Event Detection from Podcasts with People Who Stutter.* ICASSP, 2021, pp. 6798–6802.

[3] Baevski A., Zhou H., Mohamed A., Auli M. *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations.* Advances in Neural Information Processing Systems (NeurIPS), 2020, vol. 33, pp. 12449–12460.

[4] Chen S., Wang C., Chen Z. et al. *WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing.* IEEE Journal of Selected Topics in Signal Processing, 2022, vol. 16, no. 6, pp. 1505–1518.

[5] Radford A., Kim J. W., Xu T. et al. *Robust Speech Recognition via Large-Scale Weak Supervision.* International Conference on Machine Learning (ICML), 2023, pp. 28492–28518.

[6] Lugaresi C., Tang J., Nash H. et al. *MediaPipe: A Framework for Building Perception Pipelines.* CVPR Workshop, 2019.

[7] Bai S., Kolter J. Z., Koltun V. *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling.* arXiv preprint arXiv:1803.01271, 2018.

[8] Lin T. Y., Goyal P., Girshick R., He K., Dollar P. *Focal Loss for Dense Object Detection.* IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2980–2988.

Приложение А. Полные листинги программного кода

В данном приложении приведены полные исходные тексты всех разработанных программных модулей исследовательского конвейера.

А.1. Скрипт загрузки и стандартизации датасета (sep28k_data_preparation.py)

```
import os
import io
import pandas as pd
import soundfile as sf
import librosa
from datasets import load_dataset, Audio
from pathlib import Path

def download_and_prepare_sep28k():
script_dir = Path(__file__).resolve().parent
base_dir = script_dir.parent

cache_dir = base_dir / "hf_cache"
output_dir = base_dir / "data" / "sep28k_audio"

cache_dir.mkdir(parents=True, exist_ok=True)
output_dir.mkdir(parents=True, exist_ok=True)

hf_token = os.getenv("HF_TOKEN", "YOUR_HF_TOKEN")

print(f"Project root folder: {base_dir}")
print(f"Audio files will be saved to: {output_dir}")
print("Starting SEP-28k dataset download...")

dataset = load_dataset(
"saeedzou/sep28k-fluencybank-stutter-dataset",
split="train",
cache_dir=str(cache_dir),
token=hf_token
)

dataset = dataset.cast_column("audio", Audio(decode=False))

metadata = []

for i, item in enumerate(dataset):
audio_bytes = item['audio']['bytes']

audio_array, orig_sr = sf.read(io.BytesIO(audio_bytes))

if len(audio_array.shape) > 1:
audio_array = audio_array.mean(axis=1)

if orig_sr != 16000:
audio_array = librosa.resample(y=audio_array, orig_sr=orig_sr,
   target_sr=16000)

```

```
        file_name = f"clip_{i:05d}.wav"
        file_path = output_dir / file_name

        sf.write(str(file_path), audio_array, 16000)

        row = {
            'file_name': file_name,
            'prolongation': item.get('Prolongation', 0),
            'block': item.get('Block', 0),
            'sound_repetition': item.get('SoundRep', 0),
            'word_repetition': item.get('WordRep', 0),
            'interjection': item.get('Interjection', 0),
            'is_stutter': 1 if any([
            item.get('Prolongation', 0) > 0,
            item.get('Block', 0) > 0,
            item.get('SoundRep', 0) > 0,
            item.get('WordRep', 0) > 0
            ]) else 0
        }
        metadata.append(row)

        if (i + 1) % 1000 == 0:
        print(f"Processed {i + 1} files...")

        df = pd.DataFrame(metadata)
        df.to_csv(output_dir / "metadata.csv", index=False)

        print(f"Saved {len(df)} files. Metadata written to {output_dir / '
            metadata.csv'}")
        print(f"\nYou can delete the cache folder: {cache_dir}")
        print(f"Only the following folder will be used for training: {
            output_dir}")

        if __name__ == "__main__":
        download_and_prepare_sep28k()
```

A.2. Скрипт разведочного анализа данных (sep28k_data_analysis.py)

```
        import pandas as pd
        import matplotlib.pyplot as plt
        import seaborn as sns
        from pathlib import Path

        def analyze_dataset():
        script_dir = Path(__file__).resolve().parent
        base_dir = script_dir.parent
        csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"

        if not csv_path.exists():
        print(f"File {csv_path} not found!")
        return

        df = pd.read_csv(csv_path)

        print(f"Total audio fragments: {len(df)}")
```

```
18
19          class_counts = df['is_stutter'].value_counts()
20          class_percentages = df['is_stutter'].value_counts(normalize=True)
                * 100
21
22          print("\n--- Binary class balance ---")
23          print(f"Fluent speech (0): {class_counts.get(0, 0)} ({
                class_percentages.get(0, 0):.2f}%)")
24          print(f"Stutter (1):      {class_counts.get(1, 0)} ({
                class_percentages.get(1, 0):.2f}%)")
25
26          stutter_types = ['prolongation', 'block', 'sound_repetition', '
                word_repetition', 'interjection']
27          print("\n--- Occurrence of specific types (can overlap) ---")
28
29          type_stats = {}
30          for col in stutter_types:
31          count = (df[col] > 0).sum()
32          percentage = (count / len(df)) * 100
33          type_stats[col] = count
34          print(f"{col.ljust(20)}: {count} ({percentage:.2f}%)")
35
36          sns.set_theme(style="whitegrid")
37          fig, axes = plt.subplots(1, 2, figsize=(14, 6))
38
39          sns.countplot(data=df, x='is_stutter', ax=axes[0], palette=['#2
                ecc71', '#e74c3c'])
40          axes[0].set_title('Binary Class Balance (0 = Fluent, 1 = Stutter)'
                , fontsize=14)
41          axes[0].set_xlabel('Class')
42          axes[0].set_ylabel('Number of fragments')
43
44          types_s = pd.Series(type_stats).sort_values(ascending=False)
45          sns.barplot(x=types_s.values, y=types_s.index, ax=axes[1], palette
                ='viridis')
46          axes[1].set_title('Distribution of Disfluency Types', fontsize=14)
47          axes[1].set_xlabel('Number of fragments')
48
49          plt.tight_layout()
50          plt.show()
51
52          if __name__ == "__main__":
53          analyze_dataset()
```

A.3. Извлечение агрегированных признаков (extract_features.py)

```
1           import os
2           import torch
3           import pandas as pd
4           import numpy as np
5           import soundfile as sf
6           from transformers import Wav2Vec2FeatureExtractor, Wav2Vec2Model
7           from pathlib import Path
8           from tqdm import tqdm
9
```

```
        def extract_embeddings():
        script_dir = Path(__file__).resolve().parent
        base_dir = script_dir.parent
        audio_dir = base_dir / "data" / "sep28k_audio"
        csv_path = audio_dir / "metadata.csv"
        output_dir = base_dir / "data" / "sep28k_embeddings"

        output_dir.mkdir(parents=True, exist_ok=True)

        if not csv_path.exists():
        print(f"File {csv_path} not found!")
        return

        df = pd.read_csv(csv_path)

        print("Loading Wav2Vec2-XLSR-53 model...")
        model_name = "facebook/wav2vec2-large-xlsr-53"

        feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(
            model_name)
        model = Wav2Vec2Model.from_pretrained(model_name)

        device = torch.device("cuda" if torch.cuda.is_available() else "
            cpu")
        model.to(device)
        model.eval()

        print(f"Model loaded on device: {device}")
        print(f"Starting feature extraction for {len(df)} files...")

        with torch.no_grad():
        for index, row in tqdm(df.iterrows(), total=len(df)):
        file_name = row['file_name']
        file_path = audio_dir / file_name

        output_file = output_dir / file_name.replace(".wav", ".npy")
        if output_file.exists():
        continue

        if not file_path.exists():
        continue

        speech_array, sr = sf.read(str(file_path))

        inputs = feature_extractor(
        speech_array,
        sampling_rate=16000,
        return_tensors="pt",
        padding=True
        )

        inputs = {key: val.to(device) for key, val in inputs.items()}

        outputs = model(**inputs)

        hidden_states = outputs.last_hidden_state
        embedding = hidden_states.mean(dim=1).squeeze().cpu().numpy()

        np.save(str(output_file), embedding)

```

```
68          print(f"\nAll embeddings saved to: {output_dir}")
69
70          if __name__ == "__main__":
71          extract_embeddings()
```

A.4. Пофреймовая экстракция Wav2Vec2 (extract_features_seq.py)

```
1           """
2           Step 1: Extract FRAME-LEVEL embeddings from wav2vec2-large-xlsr
                -53.
3           Instead of mean pooling, we save the full T x 1024 matrix, where T
                 is the number of frames.
4           This allows the TCN classifier to detect local temporal anomalies.
5
6           Output: data/sep28k_seq_embeddings/<clip_name>.npy -- array shape
                (T, 1024)
7           """
8
9           import os
10          import torch
11          import pandas as pd
12          import numpy as np
13          import soundfile as sf
14          from transformers import Wav2Vec2FeatureExtractor, Wav2Vec2Model
15          from pathlib import Path
16          from tqdm import tqdm
17
18
19          def extract_sequential_embeddings():
20          script_dir = Path(__file__).resolve().parent
21          base_dir = script_dir.parent
22          audio_dir = base_dir / "data" / "sep28k_audio"
23          csv_path = audio_dir / "metadata.csv"
24          output_dir = base_dir / "data" / "sep28k_seq_embeddings"
25
26          output_dir.mkdir(parents=True, exist_ok=True)
27
28          if not csv_path.exists():
29          print(f"[ERROR] File {csv_path} not found!")
30          return
31
32          df = pd.read_csv(csv_path)
33
34          already_done = sum(
35          1 for _, row in df.iterrows()
36          if (output_dir / row['file_name'].replace(".wav", ".npy")).exists
                ()
37          )
38          remaining = len(df) - already_done
39          print(f"[INFO] Total files: {len(df)}, already processed: {
                already_done}, remaining: {remaining}")
40
41          if remaining == 0:
42          print("[OK] All files have been processed!")
43          return
```

```
44
45          print("[LOAD] Loading Wav2Vec2-XLSR-53...")
46          model_name = "facebook/wav2vec2-large-xlsr-53"
47          feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(
                model_name)
48          model = Wav2Vec2Model.from_pretrained(model_name)
49
50          device = torch.device("cuda" if torch.cuda.is_available() else "
                cpu")
51          model.to(device)
52          model.eval()
53
54          for param in model.parameters():
55          param.requires_grad = False
56
57          print(f"[DEVICE] {device}")
58          print(f"[START] Starting sequential extraction for {remaining}
                files...\n")
59
60          processed = 0
61          errors = 0
62
63          with torch.no_grad():
64          pbar = tqdm(df.iterrows(), total=len(df), desc="Extraction",
                colour='green')
65          for _, row in pbar:
66          file_name = row['file_name']
67          output_file = output_dir / file_name.replace(".wav", ".npy")
68
69          if output_file.exists():
70          continue
71
72          file_path = audio_dir / file_name
73          if not file_path.exists():
74          errors += 1
75          continue
76
77          try:
78          speech_array, sr = sf.read(str(file_path))
79
80          inputs = feature_extractor(
81          speech_array,
82          sampling_rate=16000,
83          return_tensors="pt",
84          padding=True
85          )
86          inputs = {key: val.to(device) for key, val in inputs.items()}
87
88          outputs = model(**inputs)
89          hidden_states = outputs.last_hidden_state  # (1, T, 1024)
90
91          seq_embedding = hidden_states.squeeze(0).cpu().numpy()
92          np.save(str(output_file), seq_embedding)
93
94          processed += 1
95
96          except Exception as e:
97          errors += 1
98          tqdm.write(f"[WARN] Error processing {file_name}: {e}")
99
```

```
        pbar.set_postfix(done=processed, err=errors)

        print(f"\n{'='*50}")
        print(f"[DONE] Extraction complete!")
        print(f"   Processed: {processed}")
        print(f"   Errors: {errors}")
        print(f"   Saved to: {output_dir}")
        print(f"{'='*50}")


        if __name__ == "__main__":
        extract_sequential_embeddings()
```

A.5. Пофреймовая экстракция WavLM (extract_features_wavlm.py)

```
        """
        Step 2: Extract frame-level embeddings from WavLM-Base-Plus.
        WavLM was trained considering paralinguistics and noise, providing
        robust features for disfluency detection.

        Output: data/sep28k_seq_wavlm/<clip_name>.npy -- array shape (T,
            768)
        """

        import torch
        import pandas as pd
        import numpy as np
        import soundfile as sf
        from transformers import WavLMModel, Wav2Vec2FeatureExtractor
        from pathlib import Path
        from tqdm import tqdm


        def extract_wavlm_embeddings():
        script_dir = Path(__file__).resolve().parent
        base_dir = script_dir.parent
        audio_dir = base_dir / "data" / "sep28k_audio"
        csv_path = audio_dir / "metadata.csv"
        output_dir = base_dir / "data" / "sep28k_seq_wavlm"

        output_dir.mkdir(parents=True, exist_ok=True)

        if not csv_path.exists():
        print(f"[ERROR] File {csv_path} not found!")
        return

        df = pd.read_csv(csv_path)

        already_done = sum(
        1 for _, row in df.iterrows()
        if (output_dir / row['file_name'].replace(".wav", ".npy")).exists
            ()
        )
        remaining = len(df) - already_done
```

```
        print(f"[INFO] Total files: {len(df)}, already processed: {already_done}, remaining: {remaining}")

        if remaining == 0:
        print("[OK] All files have been processed!")
        return

        print("[LOAD] Loading WavLM-Base-Plus...")
        model_name = "microsoft/wavlm-base-plus"

        feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(model_name)
        model = WavLMModel.from_pretrained(model_name)

        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        model.to(device)
        model.eval()

        for param in model.parameters():
        param.requires_grad = False

        print(f"[DEVICE] {device}")
        print(f"[START] Starting WavLM sequential extraction for {remaining} files...\n")

        processed = 0
        errors = 0

        with torch.no_grad():
        pbar = tqdm(df.iterrows(), total=len(df), desc="WavLM Extraction", colour='blue')
        for _, row in pbar:
        file_name = row['file_name']
        output_file = output_dir / file_name.replace(".wav", ".npy")

        if output_file.exists():
        continue

        file_path = audio_dir / file_name
        if not file_path.exists():
        errors += 1
        continue

        try:
        speech_array, sr = sf.read(str(file_path))

        inputs = feature_extractor(
        speech_array,
        sampling_rate=16000,
        return_tensors="pt",
        padding=True
        )
        inputs = {key: val.to(device) for key, val in inputs.items()}

        outputs = model(**inputs)
        hidden_states = outputs.last_hidden_state  # (1, T, 768)

        seq_embedding = hidden_states.squeeze(0).cpu().numpy()
        np.save(str(output_file), seq_embedding)
```

```
            processed += 1

        except Exception as e:
            errors += 1
            tqdm.write(f"[WARN] Error processing {file_name}: {e}")

        pbar.set_postfix(done=processed, err=errors)

    print(f"\n{'='*50}")
    print(f"[DONE] WavLM extraction complete!")
    print(f"   Processed: {processed}")
    print(f"   Errors: {errors}")
    print(f"   Saved to: {output_dir}")
    print(f"{'='*50}")


if __name__ == "__main__":
    extract_wavlm_embeddings()
```

A.6. Пофреймовая экстракция Whisper (extract_features_whisper.py)

```
"""
Step 2: Extract frame-level embeddings from Whisper encoder (small).
Whisper is a multilingual model trained on 680k hours of audio.
The encoder captures prosodic anomalies and rhythmic disruptions.

Output: data/sep28k_seq_whisper/<clip_name>.npy -- array shape (T, 768)
"""

import torch
import pandas as pd
import numpy as np
import soundfile as sf
from transformers import WhisperModel, WhisperFeatureExtractor
from pathlib import Path
from tqdm import tqdm


def extract_whisper_embeddings():
    script_dir = Path(__file__).resolve().parent
    base_dir = script_dir.parent
    audio_dir = base_dir / "data" / "sep28k_audio"
    csv_path = audio_dir / "metadata.csv"
    output_dir = base_dir / "data" / "sep28k_seq_whisper"

    output_dir.mkdir(parents=True, exist_ok=True)

    if not csv_path.exists():
        print(f"[ERROR] File {csv_path} not found!")
        return

    df = pd.read_csv(csv_path)
```

```
32
33          already_done = sum(
34          1 for _, row in df.iterrows()
35          if (output_dir / row['file_name'].replace(".wav", ".npy")).exists
                ()
36          )
37          remaining = len(df) - already_done
38          print(f"[INFO] Total files: {len(df)}, already processed: {
                already_done}, remaining: {remaining}")
39
40          if remaining == 0:
41          print("[OK] All files have been processed!")
42          return
43
44          print("[LOAD] Loading Whisper-small encoder...")
45          model_name = "openai/whisper-small"
46
47          feature_extractor = WhisperFeatureExtractor.from_pretrained(
                model_name)
48          model = WhisperModel.from_pretrained(model_name)
49
50          device = torch.device("cuda" if torch.cuda.is_available() else "
                cpu")
51          model.to(device)
52          model.eval()
53
54          for param in model.parameters():
55          param.requires_grad = False
56
57          print(f"[DEVICE] {device}")
58          print(f"[START] Starting Whisper sequential extraction for {
                remaining} files...\n")
59
60          processed = 0
61          errors = 0
62
63          with torch.no_grad():
64          pbar = tqdm(df.iterrows(), total=len(df), desc="Whisper Extraction
                ", colour='yellow')
65          for _, row in pbar:
66          file_name = row['file_name']
67          output_file = output_dir / file_name.replace(".wav", ".npy")
68
69          if output_file.exists():
70          continue
71
72          file_path = audio_dir / file_name
73          if not file_path.exists():
74          errors += 1
75          continue
76
77          try:
78          speech_array, sr = sf.read(str(file_path))
79
80          inputs = feature_extractor(
81          speech_array,
82          sampling_rate=16000,
83          return_tensors="pt",
84          padding="max_length",
85          max_length=16000 * 30,
```

```
        )
        input_features = inputs.input_features.to(device)  # (1, 80, 3000)

        encoder_outputs = model.encoder(input_features)
        hidden_states = encoder_outputs.last_hidden_state  # (1, T, 768)

        real_duration_s = len(speech_array) / 16000
        real_frames = int(np.ceil(real_duration_s * 16000 / 320))
        real_frames = min(real_frames, hidden_states.shape[1])

        seq_embedding = hidden_states[0, :real_frames, :].cpu().numpy()
        np.save(str(output_file), seq_embedding)

        processed += 1

        except Exception as e:
        errors += 1
        tqdm.write(f"[WARN] Error processing {file_name}: {e}")

        pbar.set_postfix(done=processed, err=errors)

        print(f"\n{'='*50}")
        print(f"[DONE] Whisper extraction complete!")
        print(f"   Processed: {processed}")
        print(f"   Errors: {errors}")
        print(f"   Saved to: {output_dir}")
        print(f"{'='*50}")


        if __name__ == "__main__":
        extract_whisper_embeddings()
```

A.7. Общие утилиты, функции потерь и оптимизация ОЗУ (utils.py)

```
        """
        Common utilities for the multimodal stuttering detection project.
        Contains: metrics, datasets, loss functions, constants.
        """

        import torch
        import torch.nn as nn
        import torch.nn.functional as F
        from torch.utils.data import Dataset
        import numpy as np
        import pandas as pd
        from pathlib import Path
        from tqdm import tqdm
        from sklearn.metrics import (
        f1_score, precision_score, recall_score,
        roc_auc_score, average_precision_score,
        confusion_matrix
        )

        MAX_SEQ_LEN = 150
        SAMPLE_RATE = 16000
```

```
22
23          ENCODER_DIMS = {
24              "wav2vec2": 1024,
25              "wavlm": 768,
26              "whisper": 768,
27          }
28
29          def get_project_root():
30          return Path(__file__).resolve().parent.parent
31
32
33          class FocalLoss(nn.Module):
34          def __init__(self, alpha=0.25, gamma=2.0):
35          super().__init__()
36          self.alpha = alpha
37          self.gamma = gamma
38
39          def forward(self, logits, targets):
40          bce = F.binary_cross_entropy_with_logits(logits, targets,
                reduction='none')
41          pt = torch.exp(-bce)
42          focal = self.alpha * (1 - pt) ** self.gamma * bce
43          return focal.mean()
44
45
46          class MeanPooledDataset(Dataset):
47          def __init__(self, df, embeddings_dir, desc="Loading"):
48          self.data = []
49          self.labels = []
50          df = df.reset_index(drop=True)
51          for idx in tqdm(range(len(df)), desc=f"RAM [{desc}]", leave=False)
                :
52          fname = df.loc[idx, 'file_name'].replace(".wav", ".npy")
53          emb = np.load(str(embeddings_dir / fname))
54          self.data.append(emb.astype(np.float16))
55          self.labels.append(df.loc[idx, 'is_stutter'])
56
57          def __len__(self):
58          return len(self.data)
59
60          def __getitem__(self, idx):
61          return (
62          torch.tensor(self.data[idx], dtype=torch.float32),
63          torch.tensor(self.labels[idx], dtype=torch.float32),
64          )
65
66
67          class SequentialDataset(Dataset):
68          def __init__(self, df, embeddings_dir, max_seq_len=MAX_SEQ_LEN,
                desc="Loading"):
69          self.data = []
70          self.labels = []
71          self.masks = []
72          self.max_seq_len = max_seq_len
73          df = df.reset_index(drop=True)
74
75          skipped = 0
76          for idx in tqdm(range(len(df)), desc=f"RAM [{desc}]", leave=False)
                :
77          fname = df.loc[idx, 'file_name'].replace(".wav", ".npy")
```

```
        path = embeddings_dir / fname
        if not path.exists():
        skipped += 1
        continue

        emb = np.load(str(path))  # (T, D)
        T, D = emb.shape

        if T >= max_seq_len:
        emb_padded = emb[:max_seq_len]
        mask = np.ones(max_seq_len, dtype=np.float32)
        else:
        pad_len = max_seq_len - T
        emb_padded = np.pad(emb, ((0, pad_len), (0, 0)), mode='constant')
        mask = np.concatenate([np.ones(T, dtype=np.float32),
        np.zeros(pad_len, dtype=np.float32)])

        # Store in float16 to prevent RAM overflow on 16GB systems
        self.data.append(emb_padded.astype(np.float16))
        self.masks.append(mask)
        self.labels.append(df.loc[idx, 'is_stutter'])

        if skipped > 0:
        print(f"  [WARN] Skipped {skipped} files (missing .npy)")

        def __len__(self):
        return len(self.data)

        def __getitem__(self, idx):
        return (
        torch.tensor(self.data[idx], dtype=torch.float32),
        torch.tensor(self.masks[idx], dtype=torch.float32),
        torch.tensor(self.labels[idx], dtype=torch.float32),
        )


        def compute_metrics(y_true, y_pred, y_prob=None):
        y_true = np.asarray(y_true)
        y_pred = np.asarray(y_pred)

        tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).
           ravel()
        specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0

        metrics = {
            'f1': f1_score(y_true, y_pred, zero_division=0),
            'precision': precision_score(y_true, y_pred, zero_division=0),
            'recall': recall_score(y_true, y_pred, zero_division=0),
            'specificity': specificity,
        }

        if y_prob is not None:
        y_prob = np.asarray(y_prob)
        try:
        metrics['roc_auc'] = roc_auc_score(y_true, y_prob)
        except ValueError:
        metrics['roc_auc'] = 0.0
        try:
        metrics['ap'] = average_precision_score(y_true, y_prob)
        except ValueError:
```

```
        metrics['ap'] = 0.0

        return metrics


        def print_metrics(metrics, prefix=""):
        parts = []
        for key in ['f1', 'precision', 'recall', 'specificity', 'roc_auc',
            'ap']:
        if key in metrics:
        parts.append(f"{key.upper()}: {metrics[key]:.4f}")
        print(f"{prefix}{' | '.join(parts)}")


        def metrics_to_row(metrics, method_name, **extra):
        row = {'method': method_name}
        row.update(metrics)
        row.update(extra)
        return row
```

A.8. Обучение базового классификатора MLP (train_mlp.py)

```
        import os
        import time
        import torch
        import torch.nn as nn
        import torch.optim as optim
        import torch.nn.functional as F
        from torch.utils.data import Dataset, DataLoader
        import pandas as pd
        import numpy as np
        from pathlib import Path
        from sklearn.model_selection import train_test_split
        from sklearn.metrics import classification_report, f1_score,
           precision_score, recall_score, roc_auc_score
        from tqdm import tqdm

        class FocalLoss(nn.Module):
        def __init__(self, alpha=1, gamma=2):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma

        def forward(self, inputs, targets):
        BCE_loss = F.binary_cross_entropy_with_logits(inputs, targets,
           reduction='none')
        pt = torch.exp(-BCE_loss)
        F_loss = self.alpha * (1 - pt)**self.gamma * BCE_loss
        return torch.mean(F_loss)

        class StutterDataset(Dataset):
        def __init__(self, df, embeddings_dir, desc="Loading"):
        self.df = df.reset_index(drop=True)
        self.embeddings_dir = embeddings_dir

        self.data = []
        self.labels = []
```

```
34
35          for idx in tqdm(range(len(self.df)), desc=f"To RAM [{desc}]",
                leave=False):
36          file_name = self.df.loc[idx, 'file_name']
37          npy_name = file_name.replace(".wav", ".npy")
38          embedding_path = self.embeddings_dir / npy_name
39
40          embedding = np.load(str(embedding_path))
41          label = self.df.loc[idx, 'is_stutter']
42
43          # Memory optimization: store as float16 in RAM
44          self.data.append(embedding.astype(np.float16))
45          self.labels.append(label)
46
47          def __len__(self):
48          return len(self.df)
49
50          def __getitem__(self, idx):
51          return torch.tensor(self.data[idx], dtype=torch.float32), torch.
                tensor(self.labels[idx], dtype=torch.float32)
52
53          class AudioClassifier(nn.Module):
54          def __init__(self, input_dim=1024):
55          super(AudioClassifier, self).__init__()
56          self.network = nn.Sequential(
57          nn.Linear(input_dim, 512),
58          nn.BatchNorm1d(512),
59          nn.ReLU(),
60          nn.Dropout(0.4),
61          nn.Linear(512, 128),
62          nn.BatchNorm1d(128),
63          nn.ReLU(),
64          nn.Dropout(0.3),
65          nn.Linear(128, 1)
66          )
67
68          def forward(self, x):
69          return self.network(x).squeeze(1)
70
71          def train_model():
72          script_dir = Path(__file__).resolve().parent
73          base_dir = script_dir.parent
74          embeddings_dir = base_dir / "data" / "sep28k_embeddings"
75          csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"
76
77          print("\n" + "="*60)
78          print("[INFO] Initializing data and loading metadata...")
79          print("="*60)
80
81          df = pd.read_csv(csv_path)
82
83          tqdm.pandas(desc="Checking .npy files", leave=False)
84          df['npy_exists'] = df['file_name'].progress_apply(lambda x: (
                embeddings_dir / x.replace('.wav', '.npy')).exists())
85          df = df[df['npy_exists'] == True]
86
87          train_df, val_df = train_test_split(df, test_size=0.2,
                random_state=42, stratify=df['is_stutter'])
88
```

```
        train_dataset = StutterDataset(train_df, embeddings_dir, desc="Train")
        val_dataset = StutterDataset(val_df, embeddings_dir, desc="Valid")

        train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)

        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        print(f"\n[DEVICE] Compute device: {device.type.upper()}")

        model = AudioClassifier().to(device)

        criterion = FocalLoss(alpha=0.25, gamma=2.0)
        optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-3)

        scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience=2)

        epochs = 40
        best_roc_auc = 0.0

        print("\n" + "="*60)
        print("[START] Starting MLP classifier training loop...")
        print("="*60 + "\n")

        for epoch in range(epochs):
        epoch_start_time = time.time()

        model.train()
        train_loss = 0.0

        train_loop = tqdm(train_loader, desc=f"Epoch {epoch+1:02d}/{epochs} [Train]", leave=False)

        for embeddings, labels in train_loop:
        embeddings, labels = embeddings.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(embeddings)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()
        train_loss += loss.item()

        train_loop.set_postfix(loss=f"{loss.item():.4f}")

        model.eval()
        val_preds = []
        val_probs = []
        val_true = []

        val_loop = tqdm(val_loader, desc=f"Epoch {epoch+1:02d}/{epochs} [Valid]", leave=False)

        with torch.no_grad():
        for embeddings, labels in val_loop:
```

```
        embeddings, labels = embeddings.to(device), labels.to(device)
        outputs = model(embeddings)

        probs = torch.sigmoid(outputs)
        preds = (probs > 0.5).float()

        val_probs.extend(probs.cpu().numpy())
        val_preds.extend(preds.cpu().numpy())
        val_true.extend(labels.cpu().numpy())

        val_f1 = f1_score(val_true, val_preds, zero_division=0)
        val_prec = precision_score(val_true, val_preds, zero_division=0)
        val_rec = recall_score(val_true, val_preds, zero_division=0)
        val_roc = roc_auc_score(val_true, val_probs)

        avg_train_loss = train_loss / len(train_loader)

        scheduler.step(val_roc)

        epoch_duration = time.time() - epoch_start_time
        mins, secs = divmod(epoch_duration, 60)

        print(f"Epoch {epoch+1:02d}/{epochs} [{int(mins):02d}:{int(secs)
            :02d}] | Loss: {avg_train_loss:.4f} | Val ROC-AUC: {val_roc:.4f
            } | F1: {val_f1:.4f} | Prec: {val_prec:.4f} | Rec: {val_rec:.4f
            }")

        if val_roc > best_roc_auc:
        best_roc_auc = val_roc
        torch.save(model.state_dict(), base_dir / "scripts" / "
            best_mlp_model.pth")
        print(f"  [SAVE] New best model saved! (ROC-AUC: {best_roc_auc:.4f
            })")

        print("\n" + "="*60)
        print(f"[DONE] Training complete. Best validation ROC-AUC: {
            best_roc_auc:.4f}")
        print("="*60)

        if __name__ == "__main__":
        train_model()
```

A.9. Архитектура и обучение временной сверточной сети (train_tcn.py)

```
        """
        Step 1: TCN (Temporal Convolutional Network) classifier for
            stuttering.
        Works on sequential embeddings T x 1024 from wav2vec2.

        Architecture: 4 residual dilated 1D conv blocks (dilation 1,2,4,8)
        -> Global Average Pooling -> Linear -> sigmoid.
        """

        import os
        import time
```

```
        import argparse
        import torch
        import torch.nn as nn
        import torch.optim as optim
        import torch.nn.functional as F
        from torch.utils.data import DataLoader
        import numpy as np
        import pandas as pd
        from pathlib import Path
        from sklearn.model_selection import train_test_split
        from tqdm import tqdm

        from utils import (
        FocalLoss, SequentialDataset, MAX_SEQ_LEN,
        compute_metrics, print_metrics, get_project_root
        )

        # ============================================================
        # TCN Architecture
        # ============================================================

        class ResidualBlock(nn.Module):
        def __init__(self, channels, kernel_size=3, dilation=1, dropout
            =0.2):
        super().__init__()
        padding = (kernel_size - 1) * dilation // 2

        self.conv1 = nn.Conv1d(channels, channels, kernel_size,
        padding=padding, dilation=dilation)
        self.bn1 = nn.BatchNorm1d(channels)
        self.conv2 = nn.Conv1d(channels, channels, kernel_size=1)
        self.bn2 = nn.BatchNorm1d(channels)
        self.dropout = nn.Dropout(dropout)

        def forward(self, x):
        residual = x
        out = self.dropout(F.relu(self.bn1(self.conv1(x))))
        out = self.bn2(self.conv2(out))
        if out.shape[-1] != residual.shape[-1]:
        out = out[..., :residual.shape[-1]]
        return F.relu(out + residual)


        class StutterTCN(nn.Module):
        def __init__(self, input_dim=1024, hidden_dim=256, num_blocks=4,
        kernel_size=5, dropout=0.3):
        super().__init__()

        self.input_proj = nn.Sequential(
        nn.Conv1d(input_dim, hidden_dim, kernel_size=1),
        nn.BatchNorm1d(hidden_dim),
        nn.ReLU(),
        )

        dilations = [2 ** i for i in range(num_blocks)]
        self.blocks = nn.ModuleList([
        ResidualBlock(hidden_dim, kernel_size, dilation=d, dropout=dropout
            )
        for d in dilations
        ])
```

```
69
70          self.classifier = nn.Sequential(
71          nn.Linear(hidden_dim, hidden_dim // 2),
72          nn.ReLU(),
73          nn.Dropout(dropout),
74          nn.Linear(hidden_dim // 2, 1),
75          )
76
77          def forward(self, x, mask=None):
78          x = x.permute(0, 2, 1)
79          x = self.input_proj(x)
80
81          for block in self.blocks:
82          x = block(x)
83
84          x = x.permute(0, 2, 1)
85          if mask is not None:
86          mask_expanded = mask.unsqueeze(-1)
87          x = (x * mask_expanded).sum(dim=1) / mask_expanded.sum(dim=1).
                clamp(min=1)
88          else:
89          x = x.mean(dim=1)
90
91          logits = self.classifier(x).squeeze(-1)
92          return logits
93
94
95          # ============================================================
96          # Training Loop
97          # ============================================================
98
99          def train_tcn(args):
100         base_dir = get_project_root()
101         embeddings_dir = base_dir / "data" / args.embeddings_subdir
102         csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"
103         save_path = base_dir / "scripts" / f"best_tcn_{args.encoder_name}.
                pth"
104
105         print("\n" + "=" * 60)
106         print(f"[INFO] TCN Training | Encoder: {args.encoder_name}")
107         print(f"[INFO] Embeddings: {embeddings_dir}")
108         print(f"[INFO] Input dim: {args.input_dim}")
109         print("=" * 60)
110
111         df = pd.read_csv(csv_path)
112
113         df['npy_exists'] = df['file_name'].apply(
114         lambda x: (embeddings_dir / x.replace('.wav', '.npy')).exists()
115         )
116         df = df[df['npy_exists']].drop(columns=['npy_exists'])
117         print(f"[DATA] Valid files with embeddings: {len(df)}")
118
119         train_df, val_df = train_test_split(
120         df, test_size=0.2, random_state=42, stratify=df['is_stutter']
121         )
122         print(f"[DATA] Train: {len(train_df)} | Val: {len(val_df)}")
123
124         train_dataset = SequentialDataset(train_df, embeddings_dir,
125         max_seq_len=args.max_seq_len, desc="Train")
126         val_dataset = SequentialDataset(val_df, embeddings_dir,
```

```
        max_seq_len=args.max_seq_len, desc="Valid")

        train_loader = DataLoader(train_dataset, batch_size=args.
            batch_size,
        shuffle=True, num_workers=0, pin_memory=True)
        val_loader = DataLoader(val_dataset, batch_size=args.batch_size,
        shuffle=False, num_workers=0, pin_memory=True)

        device = torch.device("cuda" if torch.cuda.is_available() else "
            cpu")
        print(f"\n[DEVICE] Compute device: {device}")

        model = StutterTCN(
        input_dim=args.input_dim,
        hidden_dim=args.hidden_dim,
        num_blocks=args.num_blocks,
        kernel_size=args.kernel_size,
        dropout=args.dropout,
        ).to(device)

        param_count = sum(p.numel() for p in model.parameters() if p.
            requires_grad)
        print(f"[MODEL] Trainable parameters: {param_count:,}")

        criterion = FocalLoss(alpha=0.25, gamma=2.0)
        optimizer = optim.AdamW(model.parameters(), lr=args.lr,
            weight_decay=args.weight_decay)
        scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=
            args.epochs, eta_min=1e-6)

        # CRITICAL FIX: Track ROC-AUC instead of F1 to avoid mode collapse
        best_roc_auc = 0.0
        patience_counter = 0

        print(f"\n{'='*60}")
        print(f"[START] Training for {args.epochs} epochs")
        print(f"{'='*60}\n")

        for epoch in range(args.epochs):
        epoch_start = time.time()

        model.train()
        train_loss = 0.0
        train_steps = 0

        pbar = tqdm(train_loader, desc=f"Epoch {epoch+1:02d}/{args.epochs}
            [Train]",
        leave=False, colour='cyan')
        for embeddings, masks, labels in pbar:
        embeddings = embeddings.to(device)
        masks = masks.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        logits = model(embeddings, masks)
        loss = criterion(logits, labels)
        loss.backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

```

```
        optimizer.step()
        train_loss += loss.item()
        train_steps += 1
        pbar.set_postfix(loss=f"{loss.item():.4f}")

        scheduler.step()

        model.eval()
        val_preds = []
        val_probs = []
        val_true = []

        with torch.no_grad():
        for embeddings, masks, labels in val_loader:
        embeddings = embeddings.to(device)
        masks = masks.to(device)

        logits = model(embeddings, masks)
        probs = torch.sigmoid(logits)
        preds = (probs > 0.5).float()

        val_probs.extend(probs.cpu().numpy())
        val_preds.extend(preds.cpu().numpy())
        val_true.extend(labels.numpy())

        metrics = compute_metrics(val_true, val_preds, val_probs)
        avg_loss = train_loss / max(train_steps, 1)
        duration = time.time() - epoch_start
        lr = optimizer.param_groups[0]['lr']

        print(f"Epoch {epoch+1:02d}/{args.epochs} [{duration:.1f}s] | "
        f"Loss: {avg_loss:.4f} | LR: {lr:.2e} | ", end="")
        print_metrics(metrics)

        # Evaluate based on ROC-AUC
        current_roc = metrics.get('roc_auc', 0.0)

        if current_roc > best_roc_auc:
        best_roc_auc = current_roc
        patience_counter = 0
        torch.save({
            'epoch': epoch + 1,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'best_roc_auc': best_roc_auc,
            'best_f1': metrics.get('f1', 0.0),
            'metrics': metrics,
            'args': vars(args),
        }, save_path)
        print(f"  [SAVE] New best model! ROC-AUC: {best_roc_auc:.4f} (F1:
            {metrics.get('f1', 0.0):.4f})")
        else:
        patience_counter += 1
        if patience_counter >= args.patience:
        print(f"\n[STOP] Early stopping: {args.patience} epochs without
            ROC-AUC improvement.")
        break

        print(f"\n{'='*60}")
```

```
        print(f"[DONE] Training complete. Best val ROC-AUC: {best_roc_auc:.4f}")
        print(f"[INFO] Model saved to: {save_path}")
        print(f"{'='*60}")

        return best_roc_auc


        def parse_args():
        parser = argparse.ArgumentParser(description="Train TCN classifier for stuttering detection")
        parser.add_argument('--encoder_name', type=str, default='wav2vec2',
        choices=['wav2vec2', 'wavlm', 'whisper'])
        parser.add_argument('--embeddings_subdir', type=str, default=None)
        parser.add_argument('--max_seq_len', type=int, default=MAX_SEQ_LEN)
        parser.add_argument('--input_dim', type=int, default=None)
        parser.add_argument('--hidden_dim', type=int, default=256)
        parser.add_argument('--num_blocks', type=int, default=4)
        parser.add_argument('--kernel_size', type=int, default=5)
        parser.add_argument('--dropout', type=float, default=0.3)
        parser.add_argument('--batch_size', type=int, default=32)
        parser.add_argument('--epochs', type=int, default=50)
        parser.add_argument('--lr', type=float, default=3e-4)
        parser.add_argument('--weight_decay', type=float, default=1e-3)
        parser.add_argument('--patience', type=int, default=10)

        args = parser.parse_args()

        from utils import ENCODER_DIMS
        if args.input_dim is None:
        args.input_dim = ENCODER_DIMS[args.encoder_name]

        if args.embeddings_subdir is None:
        subdir_map = {
            'wav2vec2': 'sep28k_seq_embeddings',
            'wavlm': 'sep28k_seq_wavlm',
            'whisper': 'sep28k_seq_whisper',
        }
        args.embeddings_subdir = subdir_map[args.encoder_name]

        return args

        if __name__ == "__main__":
        args = parse_args()
        train_tcn(args)
```

A.10. Сравнение акустических энкодеров (compare_encoders.py)

```
        """
        Step 2: Comparative metrics table across all encoders.
        Loads saved checkpoints of the best TCN models for each encoder
        and evaluates them on the validation set.

        Run: python scripts/compare_encoders.py
        """
```

```
8
9           import torch
10          import pandas as pd
11          import numpy as np
12          from pathlib import Path
13          from sklearn.model_selection import train_test_split
14          from torch.utils.data import DataLoader
15          from tabulate import tabulate
16
17          from utils import (
18          SequentialDataset, MeanPooledDataset, compute_metrics, MAX_SEQ_LEN
                ,
19          get_project_root
20          )
21          from train_tcn import StutterTCN
22
23
24          ENCODER_CONFIGS = {
25              'wav2vec2': {
26                  'embeddings_subdir': 'sep28k_seq_embeddings',
27                  'checkpoint': 'best_tcn_wav2vec2.pth',
28                  'input_dim': 1024,
29              },
30              'wavlm': {
31                  'embeddings_subdir': 'sep28k_seq_wavlm',
32                  'checkpoint': 'best_tcn_wavlm.pth',
33                  'input_dim': 768,
34              },
35              'whisper': {
36                  'embeddings_subdir': 'sep28k_seq_whisper',
37                  'checkpoint': 'best_tcn_whisper.pth',
38                  'input_dim': 768,
39              },
40          }
41
42
43          def evaluate_tcn(encoder_name, config, base_dir, device, df_val):
44          ckpt_path = base_dir / "scripts" / config['checkpoint']
45          emb_dir = base_dir / "data" / config['embeddings_subdir']
46
47          if not ckpt_path.exists():
48          print(f"  [WARN] Checkpoint {ckpt_path.name} not found - skipping
                {encoder_name}")
49          return None
50
51          if not emb_dir.exists():
52          print(f"  [WARN] Embeddings {emb_dir} not found - skipping {
                encoder_name}")
53          return None
54
55          checkpoint = torch.load(ckpt_path, map_location=device,
                weights_only=False)
56          saved_args = checkpoint.get('args', {})
57
58          model = StutterTCN(
59          input_dim=config['input_dim'],
60          hidden_dim=saved_args.get('hidden_dim', 256),
61          num_blocks=saved_args.get('num_blocks', 4),
62          kernel_size=saved_args.get('kernel_size', 5),
63          dropout=saved_args.get('dropout', 0.3),
```

```
        ).to(device)
        model.load_state_dict(checkpoint['model_state_dict'])
        model.eval()

        val_dataset = SequentialDataset(
        df_val, emb_dir,
        max_seq_len=saved_args.get('max_seq_len', MAX_SEQ_LEN),
        desc=f"Val-{encoder_name}"
        )
        val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False,
            num_workers=0)

        all_probs, all_preds, all_true = [], [], []

        with torch.no_grad():
        for embeddings, masks, labels in val_loader:
        embeddings = embeddings.to(device)
        masks = masks.to(device)
        logits = model(embeddings, masks)
        probs = torch.sigmoid(logits)
        preds = (probs > 0.5).float()

        all_probs.extend(probs.cpu().numpy())
        all_preds.extend(preds.cpu().numpy())
        all_true.extend(labels.numpy())

        metrics = compute_metrics(all_true, all_preds, all_probs)
        metrics['encoder'] = f"TCN ({encoder_name})"
        return metrics


        def evaluate_mlp(base_dir, device, df_val):
        from train_mlp import AudioClassifier

        ckpt_path = base_dir / "scripts" / "best_mlp_model.pth"
        emb_dir = base_dir / "data" / "sep28k_embeddings"

        if not ckpt_path.exists() or not emb_dir.exists():
        print("  [WARN] MLP checkpoint or embeddings not found - skipping"
            )
        return None

        model = AudioClassifier(input_dim=1024).to(device)
        model.load_state_dict(torch.load(ckpt_path, map_location=device,
            weights_only=False))
        model.eval()

        val_dataset = MeanPooledDataset(df_val, emb_dir, desc="Val-MLP")
        val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False,
            num_workers=0)

        all_probs, all_preds, all_true = [], [], []

        with torch.no_grad():
        for embeddings, labels in val_loader:
        embeddings = embeddings.to(device)
        logits = model(embeddings)
        probs = torch.sigmoid(logits)
        preds = (probs > 0.5).float()

```

```
        all_probs.extend(probs.cpu().numpy())
        all_preds.extend(preds.cpu().numpy())
        all_true.extend(labels.numpy())

        metrics = compute_metrics(all_true, all_preds, all_probs)
        metrics['encoder'] = 'MLP (wav2vec2 mean-pooled)'
        return metrics


        def main():
        base_dir = get_project_root()
        device = torch.device("cuda" if torch.cuda.is_available() else "
            cpu")

        csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"
        df = pd.read_csv(csv_path)
        _, df_val = train_test_split(
        df, test_size=0.2, random_state=42, stratify=df['is_stutter']
        )

        print("\n" + "=" * 70)
        print("  [INFO] Encoder Comparison: Unified Validation Evaluation"
            )
        print("=" * 70 + "\n")

        results = []

        print("[EVAL] MLP Baseline (mean-pooled wav2vec2)...")
        mlp_metrics = evaluate_mlp(base_dir, device, df_val)
        if mlp_metrics:
        results.append(mlp_metrics)

        for encoder_name, config in ENCODER_CONFIGS.items():
        print(f"[EVAL] TCN + {encoder_name}...")
        metrics = evaluate_tcn(encoder_name, config, base_dir, device,
            df_val)
        if metrics:
        results.append(metrics)

        if results:
        print("\n" + "=" * 70)
        print("  [RESULTS] COMPARATIVE TABLE")
        print("=" * 70 + "\n")

        df_results = pd.DataFrame(results)
        columns_order = ['encoder', 'roc_auc', 'ap', 'f1', 'precision', '
            recall', 'specificity']
        columns_present = [c for c in columns_order if c in df_results.
            columns]
        df_display = df_results[columns_present].copy()

        for col in columns_present:
        if col != 'encoder':
        df_display[col] = df_display[col].apply(lambda x: f"{x:.4f}")

        print(tabulate(df_display, headers='keys', tablefmt='grid',
            showindex=False))

        out_path = base_dir / "scripts" / "encoder_comparison.csv"
        df_results[columns_present].to_csv(out_path, index=False)
```

```
        print(f"\n[SAVE] Table saved to: {out_path}")
    else:
        print("[ERROR] No encoders were evaluated. Train the models first.")


if __name__ == "__main__":
    main()
```

A.11. Извлечение физической кинематики лица (extract_face_kinematics.py)

```
"""
Step 3: Extract facial kinematic features from video via MediaPipe Face Mesh.
Run: python scripts/extract_face_kinematics.py --video_dir <path> --output_dir <path>
"""

import argparse
import cv2
import numpy as np
import mediapipe as mp
from pathlib import Path
from tqdm import tqdm

UPPER_LIP_INNER = [82, 13, 312]
LOWER_LIP_INNER = [87, 14, 317]
LEFT_LIP_CORNER = 61
RIGHT_LIP_CORNER = 291
CHIN = 152
LEFT_JAW = 234
RIGHT_JAW = 454
LEFT_BROW_INNER = 70
RIGHT_BROW_INNER = 300
NOSE_TIP = 1
NOSE_BRIDGE = 6
LEFT_EYE_TOP = 159
LEFT_EYE_BOTTOM = 145
RIGHT_EYE_TOP = 386
RIGHT_EYE_BOTTOM = 374


def euclidean(p1, p2):
return np.sqrt(np.sum((np.array(p1) - np.array(p2)) ** 2))


def get_landmark_coords(landmarks, idx, w=1.0, h=1.0):
lm = landmarks[idx]
return np.array([lm.x * w, lm.y * h, lm.z * w])


def compute_frame_features(landmarks, w, h):
def lm(idx):
return get_landmark_coords(landmarks, idx, w, h)

```

```
        face_scale = euclidean(lm(NOSE_BRIDGE), lm(NOSE_TIP))
        if face_scale < 1e-6:
        face_scale = 1.0

        upper_lip_pts = [lm(i) for i in UPPER_LIP_INNER]
        lower_lip_pts = [lm(i) for i in LOWER_LIP_INNER]
        vertical_dist = np.mean([euclidean(u, l) for u, l in zip(
           upper_lip_pts, lower_lip_pts)])
        horizontal_dist = euclidean(lm(LEFT_LIP_CORNER), lm(
           RIGHT_LIP_CORNER))
        mar = vertical_dist / max(horizontal_dist, 1e-6)

        lip_spread = horizontal_dist / face_scale
        jaw_disp = lm(CHIN)[1] / face_scale
        brow_dist = euclidean(lm(LEFT_BROW_INNER), lm(RIGHT_BROW_INNER)) /
            face_scale

        left_brow_h = (lm(LEFT_BROW_INNER)[1] - lm(LEFT_EYE_TOP)[1]) /
           face_scale
        right_brow_h = (lm(RIGHT_BROW_INNER)[1] - lm(RIGHT_EYE_TOP)[1]) /
           face_scale
        avg_brow_height = (left_brow_h + right_brow_h) / 2

        left_eye_ap = euclidean(lm(LEFT_EYE_TOP), lm(LEFT_EYE_BOTTOM)) /
           face_scale
        right_eye_ap = euclidean(lm(RIGHT_EYE_TOP), lm(RIGHT_EYE_BOTTOM))
           / face_scale
        avg_eye_aperture = (left_eye_ap + right_eye_ap) / 2

        left_jaw_dist = euclidean(lm(LEFT_JAW), lm(CHIN))
        right_jaw_dist = euclidean(lm(RIGHT_JAW), lm(CHIN))
        jaw_asymmetry = abs(left_jaw_dist - right_jaw_dist) / face_scale

        return {
            'mar': mar,
            'lip_spread': lip_spread,
            'jaw_disp': jaw_disp,
            'brow_tension': brow_dist,
            'brow_height': avg_brow_height,
            'eye_aperture': avg_eye_aperture,
            'jaw_asymmetry': jaw_asymmetry,
        }


        def compute_temporal_derivatives(features_array, fps):
        T, N = features_array.shape
        dt = 1.0 / fps

        velocity = np.zeros_like(features_array)
        velocity[1:] = (features_array[1:] - features_array[:-1]) / dt

        acceleration = np.zeros_like(features_array)
        acceleration[1:] = (velocity[1:] - velocity[:-1]) / dt

        abs_velocity = np.abs(velocity)

        window_size = max(int(0.3 * fps), 1)
        rolling_energy = np.zeros_like(features_array)
        for i in range(T):
        start = max(0, i - window_size // 2)
```

```
        end = min(T, i + window_size // 2 + 1)
        rolling_energy[i] = np.mean(abs_velocity[start:end], axis=0)

        tremor_window = max(int(0.3 * fps), 3)
        tremor = np.zeros_like(features_array)
        for i in range(T):
        start = max(0, i - tremor_window // 2)
        end = min(T, i + tremor_window // 2 + 1)
        if end - start > 1:
        tremor[i] = np.std(velocity[start:end], axis=0)

        return np.concatenate([
        features_array, velocity, acceleration, abs_velocity,
           rolling_energy, tremor
        ], axis=1)


        def extract_video_features(video_path, target_fps=15):
        cap = cv2.VideoCapture(str(video_path))
        if not cap.isOpened():
        return None, 0

        actual_fps = cap.get(cv2.CAP_PROP_FPS)
        if actual_fps <= 0:
        actual_fps = 30.0

        w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
        frame_skip = max(1, int(round(actual_fps / target_fps)))

        mp_face_mesh = mp.solutions.face_mesh
        face_mesh = mp_face_mesh.FaceMesh(
        static_image_mode=False,
        max_num_faces=1,
        refine_landmarks=True,
        min_detection_confidence=0.5,
        min_tracking_confidence=0.5,
        )

        frame_features = []
        frame_idx = 0
        missing_frames = 0

        while True:
        ret, frame = cap.read()
        if not ret:
        break

        if frame_idx % frame_skip != 0:
        frame_idx += 1
        continue

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)

        if results.multi_face_landmarks:
        landmarks = results.multi_face_landmarks[0].landmark
        feats = compute_frame_features(landmarks, w, h)
        frame_features.append(list(feats.values()))
        else:
```

```
        frame_features.append([np.nan] * 7)
        missing_frames += 1

        frame_idx += 1

        cap.release()
        face_mesh.close()

        if len(frame_features) == 0:
        return None, actual_fps

        features_array = np.array(frame_features, dtype=np.float64)

        for col in range(features_array.shape[1]):
        mask = np.isnan(features_array[:, col])
        if mask.any() and not mask.all():
        valid_idx = np.where(~mask)[0]
        features_array[mask, col] = np.interp(
        np.where(mask)[0], valid_idx, features_array[valid_idx, col]
        )
        elif mask.all():
        features_array[:, col] = 0.0

        for col in range(features_array.shape[1]):
        median = np.median(features_array[:, col])
        q75, q25 = np.percentile(features_array[:, col], [75, 25])
        iqr = q75 - q25
        if iqr < 1e-8:
        iqr = np.std(features_array[:, col])
        if iqr < 1e-8:
        iqr = 1.0
        features_array[:, col] = np.clip((features_array[:, col] - median)
            / iqr, -6, 6)

        effective_fps = actual_fps / frame_skip
        full_features = compute_temporal_derivatives(features_array,
           effective_fps)

        if missing_frames > 0:
        total = len(frame_features)
        pct = missing_frames / total * 100
        if pct > 30:
        tqdm.write(f"  [WARN] {video_path.name}: {pct:.0f}% frames without
            detected face")

        return full_features, effective_fps


        FEATURE_NAMES_BASE = [
        'mar', 'lip_spread', 'jaw_disp', 'brow_tension',
        'brow_height', 'eye_aperture', 'jaw_asymmetry'
        ]
        DERIVATIVE_SUFFIXES = ['_raw', '_vel', '_acc', '_abs_vel', '
           _roll_energy', '_tremor']


        def get_all_feature_names():
        return [base + suffix for suffix in DERIVATIVE_SUFFIXES for base
           in FEATURE_NAMES_BASE]

```

```
210
211         def main():
212         parser = argparse.ArgumentParser(description="Extract facial
                kinematics features from video files")
213         parser.add_argument('--video_dir', type=str, required=True)
214         parser.add_argument('--output_dir', type=str, required=True)
215         parser.add_argument('--target_fps', type=int, default=15)
216         parser.add_argument('--extensions', type=str, default='.mp4,.avi,.
                mov,.mkv')
217         args = parser.parse_args()
218
219         video_dir = Path(args.video_dir)
220         output_dir = Path(args.output_dir)
221         output_dir.mkdir(parents=True, exist_ok=True)
222
223         extensions = tuple(args.extensions.split(','))
224         video_files = sorted([f for f in video_dir.iterdir() if f.suffix.
                lower() in extensions])
225
226         print(f"\n{'='*60}")
227         print(f"  [INFO] Facial Kinematics Extraction")
228         print(f"  [INFO] Video directory: {video_dir} ({len(video_files)}
                files)")
229         print(f"  [INFO] Target FPS: {args.target_fps}")
230         print(f"  [INFO] Features per frame: {len(get_all_feature_names())
                }")
231         print(f"{'='*60}\n")
232
233         processed = 0
234         for vf in tqdm(video_files, desc="Processing videos", colour='
                green'):
235         npy_path = (output_dir / vf.stem).with_suffix('.npy')
236         if npy_path.exists():
237         processed += 1
238         continue
239
240         features, _ = extract_video_features(vf, args.target_fps)
241         if features is not None:
242         np.save(str(npy_path), features.astype(np.float32))
243         processed += 1
244         tqdm.write(f"  [OK] {vf.name}: {features.shape[0]} frames x {
                features.shape[1]} features")
245         else:
246         tqdm.write(f"  [ERROR] {vf.name}: failed to process")
247
248         print(f"\n[DONE] Processed: {processed}/{len(video_files)}")
249         print(f"[INFO] Saved to: {output_dir}")
250
251
252         if __name__ == "__main__":
253         main()
```

A.12. Обучение визуального классификатора (train_face_tcn.py)

```
1         """
```

```
        Step 3: TCN classifier on visual kinematic features.
        Run: python scripts/train_face_tcn.py --features_dir <path> --
            labels_json <path>
        """

        import argparse
        import json
        import torch
        import torch.optim as optim
        from torch.utils.data import Dataset, DataLoader
        import numpy as np
        from pathlib import Path
        from sklearn.model_selection import GroupKFold
        from tqdm import tqdm

        from utils import FocalLoss, compute_metrics, get_project_root
        from train_tcn import StutterTCN


        class VideoKinematicsDataset(Dataset):
        def __init__(self, video_list, features_dir, labels_json,
        window_frames=450, stride_frames=150, desc="Loading"):
        self.data, self.labels, self.masks, self.video_ids = [], [], [],
            []
        features_dir = Path(features_dir)

        with open(labels_json, 'r') as f:
        annotations = json.load(f)

        for video_name in tqdm(video_list, desc=f"[{desc}]", leave=False):
        npy_path = features_dir / f"{video_name}.npy"
        if not npy_path.exists():
        continue

        features = np.load(str(npy_path))
        T, _ = features.shape

        frame_labels = np.zeros(T, dtype=np.float32)
        if video_name in annotations:
        for start, end in annotations[video_name]:
        frame_labels[max(0, int(start)):min(T, int(end))] = 1.0

        for win_start in range(0, T, stride_frames):
        win_end = min(win_start + window_frames, T)
        segment = features[win_start:win_end]
        seg_labels = frame_labels[win_start:win_end]

        actual_len = segment.shape[0]
        if actual_len < window_frames:
        pad_len = window_frames - actual_len
        segment = np.pad(segment, ((0, pad_len), (0, 0)), mode='constant')
        seg_labels = np.pad(seg_labels, (0, pad_len), mode='constant')
        mask = np.concatenate([
        np.ones(actual_len, dtype=np.float32),
        np.zeros(pad_len, dtype=np.float32)
        ])
        else:
        mask = np.ones(window_frames, dtype=np.float32)

```

```
        stutter_ratio = seg_labels.sum() / mask.sum() if mask.sum() > 0
            else 0
        binary_label = 1.0 if stutter_ratio >= 0.3 else 0.0

        self.data.append(segment)
        self.masks.append(mask)
        self.labels.append(binary_label)
        self.video_ids.append(video_name)

        def __len__(self):
        return len(self.data)

        def __getitem__(self, idx):
        return (
        torch.tensor(self.data[idx], dtype=torch.float32),
        torch.tensor(self.masks[idx], dtype=torch.float32),
        torch.tensor(self.labels[idx], dtype=torch.float32),
        )


        def train_face_tcn(args):
        base_dir = get_project_root()
        features_dir = Path(args.features_dir)
        save_path = base_dir / "scripts" / "best_face_tcn.pth"

        print("\n" + "=" * 60)
        print("  [INFO] Face TCN Training (Visual Kinematics)")
        print(f"  [INFO] Features: {features_dir}")
        print("=" * 60)

        with open(args.labels_json, 'r') as f:
        annotations = json.load(f)

        if args.patient_map_json:
        with open(args.patient_map_json, 'r') as f:
        patient_map = json.load(f)
        else:
        patient_map = {v: v for v in annotations.keys()}

        video_list = sorted([v for v in annotations.keys() if (
            features_dir / f"{v}.npy").exists()])
        patients = [patient_map.get(v, v) for v in video_list]

        print(f"[DATA] Videos: {len(video_list)}, Unique patients: {len(
            set(patients))}")

        device = torch.device("cuda" if torch.cuda.is_available() else "
            cpu")
        print(f"[DEVICE] Compute device: {device}")

        sample = np.load(str(features_dir / f"{video_list[0]}.npy"))
        input_dim = sample.shape[1]
        print(f"[MODEL] Input dim: {input_dim} (from {video_list[0]}.npy)"
            )

        n_folds = min(args.n_folds, len(set(patients)))
        gkf = GroupKFold(n_splits=n_folds)
        groups = np.array(patients)
        video_array = np.array(video_list)

```

```
        fold_results = []
        best_overall_roc = 0.0

        for fold, (train_idx, val_idx) in enumerate(gkf.split(video_array,
            groups=groups)):
        print(f"\n--- Fold {fold+1}/{n_folds} ---")
        train_videos = video_array[train_idx].tolist()
        val_videos = video_array[val_idx].tolist()
        print(f"  [SPLIT] Train: {len(train_videos)} videos, Val: {len(
            val_videos)} videos")

        train_dataset = VideoKinematicsDataset(
        train_videos, features_dir, args.labels_json,
        window_frames=args.window_frames, stride_frames=args.stride_frames
            ,
        desc=f"Fold-{fold+1} Train"
        )
        val_dataset = VideoKinematicsDataset(
        val_videos, features_dir, args.labels_json,
        window_frames=args.window_frames, stride_frames=args.stride_frames
            ,
        desc=f"Fold-{fold+1} Val"
        )

        if len(train_dataset) == 0 or len(val_dataset) == 0:
        print("  [WARN] Empty dataset - skipping fold")
        continue

        train_loader = DataLoader(train_dataset, batch_size=args.
            batch_size, shuffle=True, num_workers=0)
        val_loader = DataLoader(val_dataset, batch_size=args.batch_size,
            shuffle=False, num_workers=0)

        model = StutterTCN(
        input_dim=input_dim, hidden_dim=128, num_blocks=3, kernel_size=5,
            dropout=0.3
        ).to(device)

        criterion = FocalLoss(alpha=0.25, gamma=2.0)
        optimizer = optim.AdamW(model.parameters(), lr=args.lr,
            weight_decay=1e-3)
        scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=
            args.epochs, eta_min=1e-6)

        best_fold_roc = 0.0

        for epoch in range(args.epochs):
        model.train()
        for emb, mask, labels in train_loader:
        emb, mask, labels = emb.to(device), mask.to(device), labels.to(
            device)
        optimizer.zero_grad()
        logits = model(emb, mask)
        loss = criterion(logits, labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        scheduler.step()

```

```
        model.eval()
        all_probs, all_preds, all_true = [], [], []
        with torch.no_grad():
        for emb, mask, labels in val_loader:
        emb, mask = emb.to(device), mask.to(device)
        logits = model(emb, mask)
        probs = torch.sigmoid(logits)
        all_probs.extend(probs.cpu().numpy())
        all_preds.extend((probs > 0.5).float().cpu().numpy())
        all_true.extend(labels.numpy())

        metrics = compute_metrics(all_true, all_preds, all_probs)
        current_roc = metrics.get('roc_auc', 0.0)

        if current_roc > best_fold_roc:
        best_fold_roc = current_roc
        if current_roc > best_overall_roc:
        best_overall_roc = current_roc
        torch.save({
            'model_state_dict': model.state_dict(),
            'fold': fold,
            'epoch': epoch,
            'metrics': metrics,
            'input_dim': input_dim,
        }, save_path)

        print(f"  [INFO] Fold {fold+1} best ROC-AUC: {best_fold_roc:.4f}")
        fold_results.append({'fold': fold + 1, 'best_roc_auc':
           best_fold_roc})

        print(f"\n{'='*60}")
        print(f"  [DONE] Training complete")
        if fold_results:
        print(f"  [RESULTS] Mean ROC-AUC across folds: {np.mean([r['
           best_roc_auc'] for r in fold_results]):.4f}")
        print(f"  [RESULTS] Best overall ROC-AUC: {best_overall_roc:.4f}")
        if save_path.exists():
        print(f"  [SAVE] Model saved to: {save_path}")
        print(f"{'='*60}")


        def parse_args():
        parser = argparse.ArgumentParser(description="Train Face TCN on
           visual kinematics features")
        parser.add_argument('--features_dir', type=str, required=True)
        parser.add_argument('--labels_json', type=str, required=True)
        parser.add_argument('--patient_map_json', type=str, default=None)
        parser.add_argument('--window_frames', type=int, default=450)
        parser.add_argument('--stride_frames', type=int, default=150)
        parser.add_argument('--batch_size', type=int, default=16)
        parser.add_argument('--epochs', type=int, default=40)
        parser.add_argument('--lr', type=float, default=1e-3)
        parser.add_argument('--n_folds', type=int, default=6)
        return parser.parse_args()


        if __name__ == "__main__":
        args = parse_args()
        train_face_tcn(args)
```

A.13. Архитектура и обучение мультимодальной сети (train_deep_fusion.py)

```
"""
Step 4: Deep Fusion - multimodal TCN with early/deep audio-visual fusion.
Run: python scripts/train_deep_fusion.py --audio_dir <> --video_dir <> --labels_json <>
"""

import argparse
import json
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import numpy as np
from pathlib import Path
from sklearn.model_selection import GroupKFold
from tqdm import tqdm

from utils import FocalLoss, compute_metrics, get_project_root


class ResidualBlock1D(nn.Module):
def __init__(self, channels, kernel_size=5, dilation=1, dropout=0.2):
super().__init__()
padding = (kernel_size - 1) * dilation // 2
self.conv1 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)
self.bn1 = nn.BatchNorm1d(channels)
self.conv2 = nn.Conv1d(channels, channels, kernel_size=1)
self.bn2 = nn.BatchNorm1d(channels)
self.dropout = nn.Dropout(dropout)

def forward(self, x):
residual = x
out = self.dropout(F.relu(self.bn1(self.conv1(x))))
out = self.bn2(self.conv2(out))
if out.shape[-1] != residual.shape[-1]:
out = out[..., :residual.shape[-1]]
return F.relu(out + residual)


class StreamEncoder(nn.Module):
def __init__(self, input_dim, hidden_dim=256, num_blocks=4, kernel_size=5, dropout=0.3):
super().__init__()
self.proj = nn.Sequential(
nn.Conv1d(input_dim, hidden_dim, kernel_size=1),
nn.BatchNorm1d(hidden_dim),
nn.ReLU(),
)
dilations = [2 ** i for i in range(num_blocks)]
self.blocks = nn.ModuleList([
```

```
        ResidualBlock1D(hidden_dim, kernel_size, dilation=d, dropout=
            dropout)
        for d in dilations
        ])

        def forward(self, x):
        x = self.proj(x)
        for block in self.blocks:
        x = block(x)
        return x


        class DeepFusionTCN(nn.Module):
        def __init__(self, audio_dim=1024, video_dim=42, hidden_dim=256,
        audio_blocks=4, video_blocks=3, fusion_blocks=2,
        kernel_size=5, dropout=0.3, modality_dropout=0.2):
        super().__init__()
        self.modality_dropout = modality_dropout

        self.audio_encoder = StreamEncoder(audio_dim, hidden_dim,
            audio_blocks, kernel_size, dropout)
        self.video_encoder = StreamEncoder(video_dim, hidden_dim,
            video_blocks, kernel_size, dropout)

        fusion_dim = hidden_dim * 2
        self.fusion_proj = nn.Sequential(
        nn.Conv1d(fusion_dim, hidden_dim, kernel_size=1),
        nn.BatchNorm1d(hidden_dim),
        nn.ReLU(),
        )
        dilations = [2 ** i for i in range(fusion_blocks)]
        self.fusion_blocks = nn.ModuleList([
        ResidualBlock1D(hidden_dim, kernel_size, dilation=d, dropout=
            dropout)
        for d in dilations
        ])

        self.classifier = nn.Sequential(
        nn.Linear(hidden_dim, hidden_dim // 2),
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(hidden_dim // 2, 1),
        )

        def forward(self, audio, video, mask=None):
        audio = audio.permute(0, 2, 1)
        video = video.permute(0, 2, 1)

        audio_feat = self.audio_encoder(audio)
        video_feat = self.video_encoder(video)

        if self.training and self.modality_dropout > 0:
        r = torch.rand(1).item()
        if r < self.modality_dropout / 2:
        audio_feat = torch.zeros_like(audio_feat)
        elif r < self.modality_dropout:
        video_feat = torch.zeros_like(video_feat)

        fused = torch.cat([audio_feat, video_feat], dim=1)
        fused = self.fusion_proj(fused)
```

```
        for block in self.fusion_blocks:
        fused = block(fused)

        fused = fused.permute(0, 2, 1)
        if mask is not None:
        mask_exp = mask.unsqueeze(-1)
        fused = (fused * mask_exp).sum(dim=1) / mask_exp.sum(dim=1).clamp(
           min=1)
        else:
        fused = fused.mean(dim=1)

        return self.classifier(fused).squeeze(-1)


        class MultimodalDataset(Dataset):
        def __init__(self, video_list, audio_dir, video_dir, labels_json,
        target_len=450, stride=150, desc="Loading"):
        self.data_audio, self.data_video, self.labels, self.masks = [],
           [], [], []
        audio_dir = Path(audio_dir)
        video_dir = Path(video_dir)

        with open(labels_json, 'r') as f:
        annotations = json.load(f)

        for vname in tqdm(video_list, desc=f"[{desc}]", leave=False):
        audio_path = audio_dir / f"{vname}.npy"
        video_path = video_dir / f"{vname}.npy"

        if not audio_path.exists() or not video_path.exists():
        continue

        audio_feat = np.load(str(audio_path))
        video_feat = np.load(str(video_path))

        T_target = max(audio_feat.shape[0], video_feat.shape[0])
        if audio_feat.shape[0] != T_target:
        audio_feat = self._resample(audio_feat, T_target)
        if video_feat.shape[0] != T_target:
        video_feat = self._resample(video_feat, T_target)

        T = audio_feat.shape[0]
        frame_labels = np.zeros(T, dtype=np.float32)
        if vname in annotations:
        for start, end in annotations[vname]:
        frame_labels[max(0, int(start)):min(T, int(end))] = 1.0

        for ws in range(0, T, stride):
        we = min(ws + target_len, T)
        a_seg = audio_feat[ws:we]
        v_seg = video_feat[ws:we]
        l_seg = frame_labels[ws:we]

        actual = a_seg.shape[0]
        if actual < target_len:
        pad = target_len - actual
        a_seg = np.pad(a_seg, ((0, pad), (0, 0)))
        v_seg = np.pad(v_seg, ((0, pad), (0, 0)))
        l_seg = np.pad(l_seg, (0, pad))
```

```
        mask = np.concatenate([np.ones(actual), np.zeros(pad)]).astype(np.
            float32)
        else:
        mask = np.ones(target_len, dtype=np.float32)

        stutter_ratio = l_seg.sum() / mask.sum() if mask.sum() > 0 else 0
        binary_label = 1.0 if stutter_ratio >= 0.3 else 0.0

        self.data_audio.append(a_seg.astype(np.float32))
        self.data_video.append(v_seg.astype(np.float32))
        self.masks.append(mask)
        self.labels.append(binary_label)

        @staticmethod
        def _resample(arr, target_len):
        T_src, D = arr.shape
        x_src = np.linspace(0, 1, T_src)
        x_tgt = np.linspace(0, 1, target_len)
        result = np.zeros((target_len, D), dtype=arr.dtype)
        for d in range(D):
        result[:, d] = np.interp(x_tgt, x_src, arr[:, d])
        return result

        def __len__(self):
        return len(self.data_audio)

        def __getitem__(self, idx):
        return (
        torch.tensor(self.data_audio[idx]),
        torch.tensor(self.data_video[idx]),
        torch.tensor(self.masks[idx]),
        torch.tensor(self.labels[idx]),
        )


        def train_fusion(args):
        base_dir = get_project_root()
        save_path = base_dir / "scripts" / "best_deep_fusion.pth"

        print("\n" + "=" * 60)
        print("  [INFO] Deep Fusion Training (Audio + Video)")
        print("=" * 60)

        with open(args.labels_json, 'r') as f:
        annotations = json.load(f)

        if args.patient_map_json:
        with open(args.patient_map_json, 'r') as f:
        patient_map = json.load(f)
        else:
        patient_map = {v: v for v in annotations.keys()}

        audio_dir = Path(args.audio_dir)
        video_dir = Path(args.video_dir)

        video_list = sorted([
        v for v in annotations.keys()
        if (audio_dir / f"{v}.npy").exists() and (video_dir / f"{v}.npy").
            exists()
        ])
```

```
        patients = [patient_map.get(v, v) for v in video_list]

        print(f"[DATA] Videos with both modalities: {len(video_list)}")
        print(f"[DATA] Unique patients: {len(set(patients))}")

        sample_a = np.load(str(audio_dir / f"{video_list[0]}.npy"))
        sample_v = np.load(str(video_dir / f"{video_list[0]}.npy"))
        audio_dim = sample_a.shape[1]
        video_dim = sample_v.shape[1]
        print(f"[MODEL] Audio dim: {audio_dim}, Video dim: {video_dim}")

        device = torch.device("cuda" if torch.cuda.is_available() else "
            cpu")
        print(f"[DEVICE] Compute device: {device}")

        n_folds = min(args.n_folds, len(set(patients)))
        gkf = GroupKFold(n_splits=n_folds)
        groups = np.array(patients)
        video_array = np.array(video_list)

        fold_results = []
        best_overall_roc = 0.0

        for fold, (train_idx, val_idx) in enumerate(gkf.split(video_array,
             groups=groups)):
        print(f"\n--- Fold {fold+1}/{n_folds} ---")
        train_vids = video_array[train_idx].tolist()
        val_vids = video_array[val_idx].tolist()

        train_dataset = MultimodalDataset(
        train_vids, audio_dir, video_dir, args.labels_json,
        target_len=args.window_frames, stride=args.stride_frames,
        desc=f"F{fold+1} Train"
        )
        val_dataset = MultimodalDataset(
        val_vids, audio_dir, video_dir, args.labels_json,
        target_len=args.window_frames, stride=args.stride_frames,
        desc=f"F{fold+1} Val"
        )

        if len(train_dataset) == 0 or len(val_dataset) == 0:
        continue

        train_loader = DataLoader(train_dataset, batch_size=args.
            batch_size, shuffle=True, num_workers=0)
        val_loader = DataLoader(val_dataset, batch_size=args.batch_size,
            shuffle=False, num_workers=0)

        model = DeepFusionTCN(
        audio_dim=audio_dim, video_dim=video_dim,
        hidden_dim=args.hidden_dim, modality_dropout=args.modality_dropout
            ,
        ).to(device)

        criterion = FocalLoss(alpha=0.25, gamma=2.0)
        optimizer = optim.AdamW(model.parameters(), lr=args.lr,
            weight_decay=1e-3)
        scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=
            args.epochs, eta_min=1e-6)

```

```
        best_fold_roc = 0.0

        for epoch in range(args.epochs):
        model.train()
        for audio, video, mask, labels in train_loader:
        audio, video, mask, labels = audio.to(device), video.to(device),
            mask.to(device), labels.to(device)
        optimizer.zero_grad()
        logits = model(audio, video, mask)
        loss = criterion(logits, labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        scheduler.step()

        model.eval()
        all_probs, all_preds, all_true = [], [], []
        with torch.no_grad():
        for audio, video, mask, labels in val_loader:
        audio, video, mask = audio.to(device), video.to(device), mask.to(
            device)
        logits = model(audio, video, mask)
        probs = torch.sigmoid(logits)
        all_probs.extend(probs.cpu().numpy())
        all_preds.extend((probs > 0.5).float().cpu().numpy())
        all_true.extend(labels.numpy())

        metrics = compute_metrics(all_true, all_preds, all_probs)
        current_roc = metrics.get('roc_auc', 0.0)

        if current_roc > best_fold_roc:
        best_fold_roc = current_roc
        if current_roc > best_overall_roc:
        best_overall_roc = current_roc
        torch.save({
            'model_state_dict': model.state_dict(),
            'fold': fold,
            'epoch': epoch,
            'metrics': metrics,
            'audio_dim': audio_dim,
            'video_dim': video_dim,
            'hidden_dim': args.hidden_dim,
        }, save_path)

        print(f"  [INFO] Fold {fold+1} best ROC-AUC: {best_fold_roc:.4f}")
        fold_results.append(best_fold_roc)

        print(f"\n{'='*60}")
        print(f"  [DONE] Deep Fusion training complete")
        if fold_results:
        print(f"  [RESULTS] Mean ROC-AUC: {np.mean(fold_results):.4f} +/-
            {np.std(fold_results):.4f}")
        print(f"  [RESULTS] Best ROC-AUC: {best_overall_roc:.4f}")
        if save_path.exists():
        print(f"  [SAVE] Model saved to: {save_path}")
        print(f"{'='*60}")


        def parse_args():
```

```
    parser = argparse.ArgumentParser(description="Train Deep Fusion TCN")
    parser.add_argument('--audio_dir', type=str, required=True)
    parser.add_argument('--video_dir', type=str, required=True)
    parser.add_argument('--labels_json', type=str, required=True)
    parser.add_argument('--patient_map_json', type=str, default=None)
    parser.add_argument('--window_frames', type=int, default=450)
    parser.add_argument('--stride_frames', type=int, default=150)
    parser.add_argument('--hidden_dim', type=int, default=256)
    parser.add_argument('--modality_dropout', type=float, default=0.2)
    parser.add_argument('--batch_size', type=int, default=8)
    parser.add_argument('--epochs', type=int, default=50)
    parser.add_argument('--lr', type=float, default=3e-4)
    parser.add_argument('--n_folds', type=int, default=6)
    return parser.parse_args()


    if __name__ == "__main__":
    args = parse_args()
    train_fusion(args)
```

A.14. Скрипт кросс-языковой валидации и визуализации (evaluate_crosslingual.py)

```
    """
    Step 4: Cross-lingual validation - evaluating all models on Russian data.
    Run: python scripts/evaluate_crosslingual.py --russian_audio_dir <> --russian_video_dir <> --labels_json <> --output_dir <>
    """

    import argparse
    import json
    import torch
    import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    import matplotlib.patches as mpatches
    from pathlib import Path
    from tabulate import tabulate

    from utils import compute_metrics, get_project_root
    from train_tcn import StutterTCN
    from train_deep_fusion import DeepFusionTCN


    def load_frame_labels(labels_json, video_name, T):
    with open(labels_json, 'r') as f:
    annotations = json.load(f)

    frame_labels = np.zeros(T, dtype=np.float32)
    if video_name in annotations:
    for start, end in annotations[video_name]:
    s, e = max(0, int(start)), min(T, int(end))
    frame_labels[s:e] = 1.0
    return frame_labels
```

```
31
32
33          def evaluate_audio_tcn(audio_dir, labels_json, video_list, device,
                encoder_name='wav2vec2'):
34          base_dir = get_project_root()
35          ckpt_path = base_dir / "scripts" / f"best_tcn_{encoder_name}.pth"
36
37          if not ckpt_path.exists():
38          return None, None
39
40          checkpoint = torch.load(ckpt_path, map_location=device,
                weights_only=False)
41          saved_args = checkpoint.get('args', {})
42
43          model = StutterTCN(
44          input_dim=saved_args.get('input_dim', 1024),
45          hidden_dim=saved_args.get('hidden_dim', 256),
46          num_blocks=saved_args.get('num_blocks', 4),
47          kernel_size=saved_args.get('kernel_size', 5),
48          dropout=0.0,
49          ).to(device)
50          model.load_state_dict(checkpoint['model_state_dict'])
51          model.eval()
52
53          all_true, all_probs = [], []
54          per_video_results = {}
55          window_len = saved_args.get('max_seq_len', 150)
56          stride = window_len // 2
57          audio_dir = Path(audio_dir)
58
59          for vname in video_list:
60          npy_path = audio_dir / f"{vname}.npy"
61          if not npy_path.exists():
62          continue
63
64          features = np.load(str(npy_path))
65          T = features.shape[0]
66          frame_labels = load_frame_labels(labels_json, vname, T)
67
68          frame_scores = np.zeros(T)
69          frame_counts = np.zeros(T)
70
71          with torch.no_grad():
72          for ws in range(0, T, stride):
73          we = min(ws + window_len, T)
74          segment = features[ws:we]
75          actual = segment.shape[0]
76
77          if actual < window_len:
78          segment = np.pad(segment, ((0, window_len - actual), (0, 0)))
79          mask = np.concatenate([np.ones(actual), np.zeros(window_len -
                actual)])
80          else:
81          mask = np.ones(window_len)
82
83          seg_t = torch.tensor(segment, dtype=torch.float32).unsqueeze(0).to
                (device)
84          mask_t = torch.tensor(mask, dtype=torch.float32).unsqueeze(0).to(
                device)
85
```

```
        logit = model(seg_t, mask_t)
        prob = torch.sigmoid(logit).item()

        frame_scores[ws:we] += prob
        frame_counts[ws:we] += 1

        frame_counts = np.maximum(frame_counts, 1)
        frame_probs = frame_scores / frame_counts

        all_true.extend(frame_labels.tolist())
        all_probs.extend(frame_probs.tolist())
        per_video_results[vname] = {'labels': frame_labels, 'probs':
            frame_probs}

    all_preds = [1 if p > 0.5 else 0 for p in all_probs]
    metrics = compute_metrics(all_true, all_preds, all_probs)
    return metrics, per_video_results


def evaluate_deep_fusion(audio_dir, video_dir, labels_json,
    video_list, device):
    base_dir = get_project_root()
    ckpt_path = base_dir / "scripts" / "best_deep_fusion.pth"

    if not ckpt_path.exists():
        return None, None

    checkpoint = torch.load(ckpt_path, map_location=device,
        weights_only=False)

    model = DeepFusionTCN(
    audio_dim=checkpoint['audio_dim'],
    video_dim=checkpoint['video_dim'],
    hidden_dim=checkpoint.get('hidden_dim', 256),
    modality_dropout=0.0,
    ).to(device)
    model.load_state_dict(checkpoint['model_state_dict'])
    model.eval()

    audio_dir = Path(audio_dir)
    video_dir = Path(video_dir)

    all_true, all_probs = [], []
    per_video_results = {}
    window_len = 450
    stride = 150

    for vname in video_list:
        a_path = audio_dir / f"{vname}.npy"
        v_path = video_dir / f"{vname}.npy"
        if not a_path.exists() or not v_path.exists():
            continue

        audio_feat = np.load(str(a_path))
        video_feat = np.load(str(v_path))

        T = max(audio_feat.shape[0], video_feat.shape[0])
        if audio_feat.shape[0] != T:
            audio_feat = _resample_array(audio_feat, T)
        if video_feat.shape[0] != T:
```

```
        video_feat = _resample_array(video_feat, T)

        frame_labels = load_frame_labels(labels_json, vname, T)
        frame_scores = np.zeros(T)
        frame_counts = np.zeros(T)

        with torch.no_grad():
        for ws in range(0, T, stride):
        we = min(ws + window_len, T)
        a_seg = audio_feat[ws:we]
        v_seg = video_feat[ws:we]
        actual = a_seg.shape[0]

        if actual < window_len:
        pad = window_len - actual
        a_seg = np.pad(a_seg, ((0, pad), (0, 0)))
        v_seg = np.pad(v_seg, ((0, pad), (0, 0)))
        mask = np.concatenate([np.ones(actual), np.zeros(pad)])
        else:
        mask = np.ones(window_len)

        a_t = torch.tensor(a_seg, dtype=torch.float32).unsqueeze(0).to(
            device)
        v_t = torch.tensor(v_seg, dtype=torch.float32).unsqueeze(0).to(
            device)
        m_t = torch.tensor(mask, dtype=torch.float32).unsqueeze(0).to(
            device)

        logit = model(a_t, v_t, m_t)
        prob = torch.sigmoid(logit).item()

        frame_scores[ws:we] += prob
        frame_counts[ws:we] += 1

        frame_counts = np.maximum(frame_counts, 1)
        frame_probs = frame_scores / frame_counts

        all_true.extend(frame_labels.tolist())
        all_probs.extend(frame_probs.tolist())
        per_video_results[vname] = {'labels': frame_labels, 'probs':
            frame_probs}

        all_preds = [1 if p > 0.5 else 0 for p in all_probs]
        metrics = compute_metrics(all_true, all_preds, all_probs)
        return metrics, per_video_results


        def _resample_array(arr, target_len):
        T_src, D = arr.shape
        x_src = np.linspace(0, 1, T_src)
        x_tgt = np.linspace(0, 1, target_len)
        result = np.zeros((target_len, D), dtype=arr.dtype)
        for d in range(D):
        result[:, d] = np.interp(x_tgt, x_src, arr[:, d])
        return result


        def plot_timeline(video_name, labels, probs, threshold=0.5,
            save_path=None):
        fig, ax = plt.subplots(1, 1, figsize=(14, 3))
```

```
        T = len(labels)
        x = np.arange(T)

        in_event = False
        for i in range(T):
        if labels[i] > 0.5 and not in_event:
        start = i
        in_event = True
        elif (labels[i] <= 0.5 or i == T - 1) and in_event:
        ax.axvspan(start, i, alpha=0.3, color='green')
        in_event = False

        preds = (np.array(probs) > threshold).astype(float)
        in_det = False
        for i in range(T):
        if preds[i] > 0.5 and not in_det:
        start = i
        in_det = True
        elif (preds[i] <= 0.5 or i == T - 1) and in_det:
        ax.axvspan(start, i, alpha=0.2, color='red', hatch='///')
        in_det = False

        ax.plot(x, probs, color='blue', linewidth=0.8, alpha=0.8)
        ax.axhline(y=threshold, color='gray', linestyle='--', alpha=0.5,
            linewidth=0.5)

        ax.set_xlim(0, T)
        ax.set_ylim(0, 1)
        ax.set_xlabel('Frame')
        ax.set_ylabel('Score')
        ax.set_title(f'{video_name}', fontsize=11)

        handles = [
        mpatches.Patch(color='green', alpha=0.3, label='Annotation'),
        mpatches.Patch(color='red', alpha=0.2, hatch='///', label='
            Detection'),
        ]
        ax.legend(handles=handles, loc='upper right', fontsize=8)
        plt.tight_layout()

        if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        plt.close()
        else:
        plt.show()


        def main():
        parser = argparse.ArgumentParser(description="Cross-lingual
            evaluation on Russian data")
        parser.add_argument('--russian_audio_dir', type=str, required=True
            )
        parser.add_argument('--russian_video_dir', type=str, default=None)
        parser.add_argument('--labels_json', type=str, required=True)
        parser.add_argument('--output_dir', type=str, default=None)
        parser.add_argument('--encoders', type=str, default='wav2vec2,
            wavlm,whisper')
        args = parser.parse_args()

```

```
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

        with open(args.labels_json, 'r') as f:
        annotations = json.load(f)
        video_list = sorted(annotations.keys())

        print("\n" + "=" * 70)
        print("  [INFO] Cross-Lingual Validation (RU)")
        print(f"  [INFO] Videos: {len(video_list)}")
        print("=" * 70)

        results = []
        all_per_video = {}

        for enc in args.encoders.split(','):
        enc = enc.strip()
        print(f"\n[EVAL] Audio TCN ({enc})...")
        metrics, per_vid = evaluate_audio_tcn(
        args.russian_audio_dir, args.labels_json, video_list, device, enc
        )
        if metrics:
        metrics['method'] = f'TCN ({enc})'
        results.append(metrics)
        all_per_video[f'TCN_{enc}'] = per_vid
        print(f"  ROC-AUC: {metrics['roc_auc']:.4f} | F1: {metrics['f1']:.4f} | "
        f"Prec: {metrics['precision']:.4f} | Rec: {metrics['recall']:.4f} | Spec: {metrics['specificity']:.4f}")
        else:
        print(f"  [WARN] Model not found")

        if args.russian_video_dir:
        print(f"\n[EVAL] Deep Fusion (audio + video)...")
        metrics, per_vid = evaluate_deep_fusion(
        args.russian_audio_dir, args.russian_video_dir,
        args.labels_json, video_list, device
        )
        if metrics:
        metrics['method'] = 'Deep Fusion'
        results.append(metrics)
        all_per_video['DeepFusion'] = per_vid
        print(f"  ROC-AUC: {metrics['roc_auc']:.4f} | F1: {metrics['f1']:.4f} | "
        f"Prec: {metrics['precision']:.4f} | Rec: {metrics['recall']:.4f} | Spec: {metrics['specificity']:.4f}")

        if results:
        print("\n" + "=" * 70)
        print("  [RESULTS] SUMMARY TABLE (Russian Data)")
        print("=" * 70 + "\n")

        df = pd.DataFrame(results)
        cols = ['method', 'roc_auc', 'ap', 'f1', 'precision', 'recall', 'specificity']
        cols = [c for c in cols if c in df.columns]
        df_disp = df[cols].copy()
        for c in cols:
        if c != 'method':
        df_disp[c] = df_disp[c].apply(lambda x: f"{x:.4f}")
```

```
306         print(tabulate(df_disp, headers='keys', tablefmt='grid', showindex
                =False))
307
308         print("\n  [INFO] Laboratory Baselines (for reference):")
309         print("  Vocametrix:     ROC-AUC=0.701 | F1=0.487 | Prec=0.327 |
                Rec=0.957 | Spec=0.300")
310         print("  StutterLens:    ROC-AUC=0.632 | F1=0.269 | Prec=0.391 |
                Rec=0.205 | Spec=0.886")
311         print("  Lab Late Fus.:  ROC-AUC=0.710 | F1=0.282 | Prec=0.504 |
                Rec=0.195 | Spec=0.929")
312
313         out_path = get_project_root() / "scripts" / "crosslingual_results.
                csv"
314         df[cols].to_csv(out_path, index=False)
315         print(f"\n  [SAVE] Results saved to: {out_path}")
316
317         if args.output_dir and all_per_video:
318         output_dir = Path(args.output_dir)
319         output_dir.mkdir(parents=True, exist_ok=True)
320         print(f"\n  [INFO] Generating timeline plots...")
321
322         for method_key, per_vid in all_per_video.items():
323         if per_vid is None:
324         continue
325         for vname, data in per_vid.items():
326         plot_path = output_dir / f"timeline_{method_key}_{vname}.png"
327         plot_timeline(
328         f"{vname} ({method_key})",
329         data['labels'], data['probs'],
330         save_path=str(plot_path)
331         )
332         print(f"  [DONE] Plots saved to: {output_dir}")
333
334
335         if __name__ == "__main__":
336         main()
```